

Math 151A - Applied Numerical Methods

Angad Kapoor

Contents

1	Algorithms and Convergence	2
2	Root-Finding Algorithms	5
2.1	Bisection Method	5
2.2	Fixed Point Algorithms	7
2.2.1	Fixed-Point Method (FPM)	7
2.2.2	Fixed-Point Problems (FPP)	7
2.3	Newton's Method	13
2.4	Secant Method	14
3	Error Analysis	15
4	Interpolation	19
4.1	Lagrange Polynomials	19
4.1.1	Error Bounds for Lagrange Polynomials	20
4.2	Data Approximation and Neville's Method	21
4.3	Divided Differences	21
5	Numerical Differentiation	23
5.1	Method of Undetermined Coefficients	26
5.2	2nd Order Finite Differences	28
5.3	Imaging - Edge Detection	29
6	Numerical Integration	31
6.1	Closed Quadrature Rules	31
6.2	Open Quadrature Rules	35
6.3	Composite Quadrature Rules	37
6.4	Composite Midpoint Rule	39
6.5	Gaussian Quadrature	40

7	Linear Algebra	43
7.1	Norms	43
7.2	Eigenvalues and Eigenvectors	43
7.3	Power Method	44
7.3.1	Rayleigh Quotient	44
7.3.2	Power Iteration	45
7.3.3	Deflation	46
7.4	Linear Models	46
7.5	Non-linear Models	48

1 Algorithms and Convergence

Definition. Suppose $\{\beta_n\}_{n=0}^{\infty}$ converges to zero, and $\{x_n\}_{n=0}^{\infty}$ converges to some value $\bar{x}$, and $\exists$ constant $c > 0$ such that $|x_n - \bar{x}| \leq c\beta_n$ for $n \gg 1$.

Then, we say that $x_n \rightarrow \bar{x}$ (as $n \rightarrow \infty$) at rate $\mathcal{O}(\beta_n)$.

Short hand for this: $x_n = \bar{x} + \mathcal{O}(\beta_n)$

Although any sequence β_n can be used for the rate, we usually try to use $\beta_n = 1/n^p$, and try to find the largest value of p .

To find this rate β_n , we can use **Taylor's Theorem**. Using a couple of attempts (ie. guesswork) for the cutoff point, the Taylor polynomial derived is used for the limit, while the remainder polynomial is used for calculating the rate and $\mathcal{O}$.

Taylor's Theorem. Consider $f \in C^{m+1}$ over $[a, b]$ (ie. the $(m + 1)^{\text{th}}$ derivative of f exists and is continuous on $[a, b]$). Also, we have a centered point $x_0 \in [a, b]$.

For every $x \in [a, b]$, $\exists$ an unknown function $\xi(x)$ between x and x_0 with $f(x) = P_m(x) + R_m(x)$.

$$P_m(x) = \sum_{n=0}^m \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

$$R_m(x) = \frac{f^{(m+1)}(\xi(x))}{(m+1)!} (x - x_0)^{m+1}$$

This can also be defined in terms of functions.

Definition. Suppose $\lim_{h \rightarrow 0} G(h) = 0$, and $\lim_{h \rightarrow 0} F(h) = L$.

If $\exists$ a constant $c > 0$ such that $|F(h) - L| \leq c \cdot G(h)$ for a small enough h , then we say $F(h) \rightarrow L$ at the rate $G(h)$, or:

$$F(h) = L + \mathcal{O}(G(h))$$

Example: $\sin(1/n)$

Let $x_n = 1/n$, and $y_n = \sin(x_n) = \sin(1/n)$. Evaluate at $x_0 = \bar{x} = 0$.

Hence, as $n \rightarrow \infty$, $x_n \rightarrow 0 \implies \bar{x} = 0$ and $\bar{y} = 0$

Using Taylor's Theorem (we will expand till one term only):

$$\begin{aligned}
 y_n = \sin(x_n) &= \mathbf{Taylor}(\sin(x_n)) \\
 &= P(x_n) + R(x_n) \\
 &= \sin(\bar{x}) + \frac{d}{dx} \sin(x) \Big|_{x=\xi(x)} (x_n - \bar{x}) \\
 &= \sin(\bar{x}) + \cos(\xi(x)) (x_n - \bar{x}) \\
 &= \sin(0) + \cos(\xi(x)) \left(\frac{1}{n} - 0 \right) \\
 \therefore y_n &= \frac{1}{n} \cos(\xi(x))
 \end{aligned}$$

From the earlier theorem, we have:

$$|x_n - \bar{x}| \leq c\beta_n$$

Using $y_n = \sin(1/n)$ and $\bar{y} = 0$:

$$\begin{aligned}
 |\sin(x_n) - 0| &= \left| \frac{1}{n} \cos(\xi(x)) \right| \\
 &= |\cos(\xi(x))| \cdot \frac{1}{n} \\
 &\leq 1 \cdot \frac{1}{n} \\
 &= \frac{1}{n}
 \end{aligned}$$

Hence, as $n \rightarrow \infty$, $\sin(1/n) \rightarrow 0$ at rate $\frac{1}{n}$:

$$\sin \frac{1}{n} = 0 + \mathcal{O}\left(\frac{1}{n}\right)$$

Note: using further expanded terms of $\sin x$ will give another constant c , but the rate β_n will still be $1/n$.

Example: $\frac{1-e^h}{h}$

We know $\lim_{h \rightarrow 0} \frac{1 - e^h}{h} = -1$.

This can be simplified using the Taylor expansion since $e^x \in C^\infty(\mathbb{R})$.

Use $x_0 = 0$ as the centering point for e^x .

$$\begin{aligned}
e^x &= P(x) + R(x) \\
&= e^{x_0} + \frac{d}{dx}e^x \Big|_{x=x_0} (x - x_0) + \frac{d^2}{dx^2}e^x \Big|_{x=\xi(x)} (x - x_0)^2 \\
&= e^{x_0} + e^{x_0}(x - x_0) + e^{\xi(x)} \frac{(x - x_0)^2}{2!} \\
&= e^0 + e^0(x - 0) + e^{\xi(x)} \frac{(x - 0)^2}{2!} \\
&= 1 + x + e^{\xi(x)} \frac{x^2}{2}
\end{aligned}$$

Rewriting in terms of h :

$$\begin{aligned}
e^h &= 1 + h + e^{\xi(h)} \frac{h^2}{2} \\
\therefore \frac{1 - e^h}{h} &= \frac{1 - \left(1 + h + e^{\xi(h)} \frac{h^2}{2}\right)}{h} \\
&= \frac{-h - e^{\xi(h)} \frac{h^2}{2}}{h} \\
&= -1 - e^{\xi(h)} \frac{h}{2}
\end{aligned}$$

Applying the definition from earlier:

$$\begin{aligned}
|F(h) - L| &= \left| \frac{1 - e^h}{h} + 1 \right| \\
&= \left| e^{\xi(h)} \frac{h}{2} \right|
\end{aligned}$$

Now, we know that by definition, $\xi(h)$ is between 0 and h (technically, h and x_0). Also, the definition from earlier is valid for a small enough h . So we can for this question set $0 \leq h \leq 1$, which leads to $\xi(h) \leq 1$.

Hence:

$$\begin{aligned}
\left| e^{\xi(h)} \frac{h}{2} \right| &\leq \frac{e^1 \cdot h}{2} \\
&= \frac{e}{2} h \\
&\leq \frac{3}{2} h
\end{aligned}$$

Hence, the rate is $\frac{3}{2}h$.

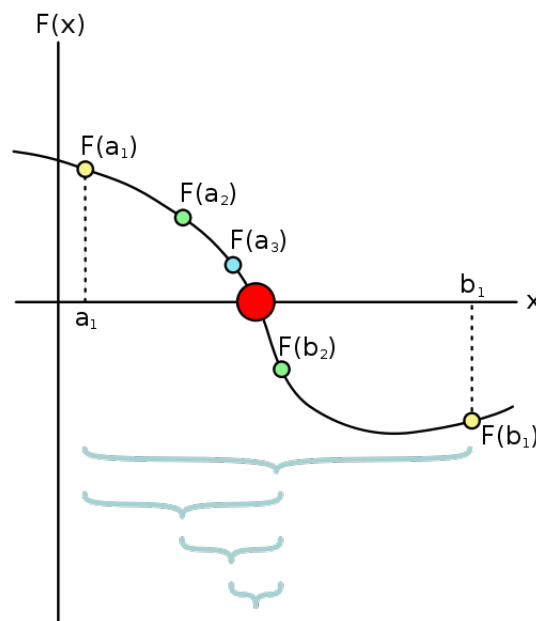
$$\frac{1 - e^h}{h} = -1 + \mathcal{O}\left(\frac{3}{2}h\right)$$

2 Root-Finding Algorithms

Aim: to find an $x \in \mathbb{R}$ such that $f(x) = 0$ for a given f (maybe also given $[a, b]$).
Some conditions that need to be satisfied:

- i. f is continuous on the given interval $[a, b]$
- ii. $f(a)$ and $f(b)$ have different signs
- iii. (this might be relaxed later in course) there is a unique root ie. only one root in $[a, b]$

2.1 Bisection Method



Method:

1. to begin, set $a_1 = a$ and $b_1 = b$, and let p_1 be the midpoint of $[a_1, b_1]$:

$$p_1 = \frac{a_1 + b_1}{2} = a_1 + \frac{b_1 - a_1}{2}$$

2. if $f(p_1) = 0$, then $p = p_1$ and we are done

3. if $f(p_1) \neq 0$:

- if $f(p_1)$ and $f(a_1)$ have the same sign, then $p \in (p_1, b_1)$. Set $a_1 = p_1$ and $b_2 = b_1$

- if $f(p_1)$ and $f(a_1)$ have opposite signs, then $p \in (a_1, p_1)$. Set $a_2 = a_1$ and $b_2 = p_1$

4. Repeat until root is found or tolerance threshold is reached

Theorem. If $f \in C$ over $[a, b]$ and $\text{sgn}(a) \times \text{sgn}(b) < 0$, and the root p is unique on $[a, b]$, then the sequence generated by the bisection algorithm denoted by p_n has the error:

$$|p_n - p| \leq \frac{b - a}{2^n}, \quad \forall n \geq 1$$

Proof. Let L_n be the length of the interval after step n . We have that for any $n \geq 2$:

$$\begin{aligned} L_n &= \frac{1}{2} L_{n-1} \\ &= \frac{1}{2^2} L_{n-2} \\ &= \frac{1}{2^3} L_{n-3} \\ &\vdots \\ &= \frac{1}{2^{n-1}} L_{n-1} \\ \therefore L_n &= \frac{b - a}{2^{n-1}} \end{aligned}$$

We know that $p \in [a_n, b_n]$, and p_n is always the midpoint $\frac{a_n + b_n}{2}$, which is where the extra $\frac{1}{2}$ factor comes from.

$$\begin{aligned} \text{Error} = |p_n - p| &\leq \max\{|a_n - p_n|, |b_n - p_n|\} \\ &= \frac{L_n}{2} = \frac{b - a}{2^n} \end{aligned}$$

□

Hence:

$$p_n = p + \mathcal{O}\left(\frac{1}{2^n}\right)$$

$$\begin{aligned} \lim_{n \rightarrow \infty} E_n &= \lim_{n \rightarrow \infty} |p_n - p| \\ &\leq \lim_{n \rightarrow \infty} \frac{b - a}{2^n} \\ &= 0 \end{aligned}$$

Tolerance Threshold: If we want the error to be bounded by a tolerance value $\epsilon > 0$, then we can find N such that:

$$|p_N - p| \leq \frac{b - a}{2^N} \leq \epsilon$$

$$\begin{aligned} \therefore \frac{b-a}{2^N} &\leq \epsilon \\ \frac{b-a}{\epsilon} &\leq 2^N \\ \therefore \log_2 \left(\frac{b-a}{\epsilon} \right) &\leq N \end{aligned}$$

Hence, taking N steps of the bisection algorithm with $N \geq \log_2 \left(\frac{b-a}{\epsilon} \right)$ guarantees an ϵ -accurate approximation of the root p .

(Of course, it is possible for the algorithm to produce an ϵ -accurate approximation in $\leq N$ steps.)

2.2 Fixed Point Algorithms

When we design an algorithm to compute a solution to a non-linear problem, we often try to write it as a fixed-point problem and apply a fixed-point method to solve it.

2.2.1 Fixed-Point Method (FPM)

Generates a sequence $\{p_n\}_{n=1}^{\infty}$ by iterating over the loop, with a known p_0

```

for  $n = 1, 2, \dots$  do
  |  $p_n = g(p_{n-1})$ 
end

```

Algorithm 1: Fixed-Point Method

If the sequence, converges, it will converge to a fixed point ie. $p_n \rightarrow p^*$ and $p_{n-1} \rightarrow p^*$ as $n \rightarrow \infty$

2.2.2 Fixed-Point Problems (FPP)

Aim: to find an $x \in \mathbb{R}$ such that $g(x) = x$. The point p such that $g(p) = p$ is called a “Fixed Point”.

A root-finding problem can be made into an FPP to solve: $f(x) = 0 \implies x + f(x) = x$. Simply set $g(x) = x + f(x)$, and use that.

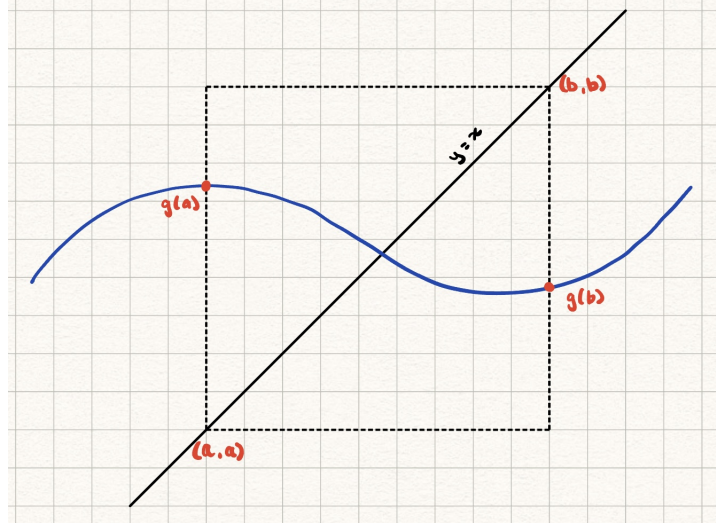
However, one can rearrange $f(x) = 0$ in multiple ways that can result in multiple different $g(x)$. To choose the correct one, that gives the most efficient answer, we can implement the existence and uniqueness theorem.

Theorem. (Banach’s Fixed Point Theorem) If $g \in C^{[a,b]}$ and $g(x) : [a, b] \mapsto [a, b]$, then there exists a fixed point $p \in [a, b]$. In addition, if $g'(x)$ exists on $[a, b]$ and $\exists K \in (0, 1)$ with $|g'(x)| \leq K < 1 \forall x \in [a, b]$, then the found fixed point is unique.

Note: the condition that $g(x) \in [a, b] \forall x \in [a, b]$ corresponds to g entering the box $[a, b] \times [a, b]$ from the left and exiting from the right - this ensures an intersection with $y = x$

Proof. If $g(a) = a$ or $g(b) = b$, then the first part holds.

Assume that $g(a) \neq a$ and $g(b) \neq b$. From the diagram below, it can be seen that $g(a) > a$ and $g(b) < b$



Define $h(x) = g(x) - x$. Since $g \in C^{[a,b]} \implies h \in C^{[a,b]}$.

At $x = a$, $h(a) = g(a) - a > 0$ and at $x = b$, $h(b) = g(b) - b < 0$ (from our assumption). Hence, by the Intermediate Value Theorem, $\exists p \in [a, b]$ such that $h(p) = 0$.

$\therefore 0 = h(p) = g(p) - p \implies g(p) = p \implies p \in [a, b]$ is a fixed point for g .

Now the prove that p is a unique fixed point, assume there exist two distinct fixed points p and q for g in $[a, b]$ ie. $g(p) = p$ and $g(q) = q$ but $p \neq q$. By the Mean Value Theorem, $\exists \lambda \in [a, b]$ such that:

$$\frac{g(p) - g(q)}{p - q} = g'(\lambda)$$

So,

$$\begin{aligned} |p - q| &= |g(p) - g(q)| \quad \dots (\text{Fixed Points}) \\ &= |g'(\lambda) \cdot (p - q)| \quad \dots (\text{MVT}) \\ &= |g'(\lambda)| \cdot |p - q| \\ &\leq K|p - q| \\ &< |p - q| \end{aligned}$$

This provides a contradiction, since $|p - q| \not\leq |p - q|$, and hence, p and q are not distinct. $\square$

Theorem. (Convergence of the Fixed-Point Method) Let $g \in C'[a, b]$ (ie. $g'(x)$ exists on $[a, b]$) such that $g : [a, b] \mapsto [a, b]$, and $\exists K \in (0, 1)$ such that $|g'(x)| \leq K \forall x \in [a, b]$. Then, for any $p_0 \in [a, b]$, the sequence $p_n = g(p_{n-1})$ converges to the unique fixed point p with $g(p) = p$.

Proof. Bu the assumption, applying the previous proof gives us: $\exists$ a unique fixed point $p \in [a, b]$ with $p = g(p)$.

By the Mean Value Theorem, $\exists \xi_{n-1} \in [a, b]$ such that:

$$\frac{g(p_{n-1}) - g(p)}{p_{n-1} - p} = g'(\xi_{n-1})$$

We want to show that $E_n = |p_n - p|$ goes to zero as $n \rightarrow \infty$.

$$\begin{aligned} E_n &= |p_n - p| = |g(p_{n-1}) - g(p)| \\ &= |g'(\xi_{n-1}) \cdot (p_{n-1} - p)| \\ &= |g'(\xi_{n-1})| \cdot |p_{n-1} - p| \\ &= |g'(\xi_{n-1})| E_{n-1} \\ &\leq K \cdot E_{n-1} \\ &\leq K^2 \cdot E_{n-2} \\ &\vdots \\ &\leq K^n E_0 \end{aligned}$$

We know that $K \in (0, 1)$, hence $K^n \leq K$.

$$\begin{aligned} \lim_{n \rightarrow \infty} E_n &\leq \lim_{n \rightarrow \infty} K^n E_0 \\ &= E_0 \lim_{n \rightarrow \infty} K^n \\ &= 0 \end{aligned}$$

Hence, as $n \rightarrow \infty$, $E_n \rightarrow 0$.

$$\therefore \lim_{n \rightarrow \infty} |p_n - p| = 0 \implies p_n \rightarrow p$$

□

Note: for existence and uniqueness, $|g'(x)| \leq K$ is sufficient but not necessary (using just $g'(x) \leq K$ also works). But for converge, $|g'(x)| \leq K$ is *required*.

Corollary. (Error Bound for FPM) Let $g \in C^1[a, b]$, $g : [a, b] \mapsto [a, b]$ and $\exists K \in (0, 1) : |g'(x)| \leq K \forall x \in [a, b]$

Let p be the unique fixed point, and p_n be generated by the FPM $p_n = g(p_{n-1})$. Then, the error can be bounded by:

a. $|p_n - p| \leq K^n \max\{p_0 - a, b - p_0\}$

b. $|p_n - p| \leq \frac{K^n}{1 - K} |p_0 - p_1|$

Proof. Proving each bound:

- a. From earlier, we know that $E_n \leq K^n E_0$ (since g and $[a, b]$ both satisfy the conditions required). Thus, $|p_n - p| \leq K^n |p_0 - p|$. We know that $p \in [a, b] = [a, p_0] \cup [p_0, b]$. This gives:

$$|p_n - p| \leq K^n \max\{p_0 - a, b - p_0\}$$

- b. The idea of this proof is to consider the error based on some $m > n$, and then let $m \rightarrow \infty$:

$$E_n = |p_n - p| = \lim_{m \rightarrow \infty} |p_n - p_m|$$

We also know that $|p_{n+1} - p_n| = |g(p_{n+1}) - g(p_n)|$. Hence, by the Mean Value Theorem:

$$\begin{aligned} |p_{n+1} - p_n| &= |g(p_{n+1}) - g(p_n)| \\ &\leq K |p_n - p_{n-1}| \\ &\vdots \\ &\leq K^n |p_1 - p_0| \end{aligned}$$

To get a bound on $|p_n - p_m|$, $m > n$, we can use a telescoping sum:

$$\begin{aligned} |p_m - p_n| &= |(p_m - p_{m-1}) + (p_{m-1} - p_{m-2}) + \cdots + (p_{n+1} - p_n)| \\ &\leq |p_m - p_{m-1}| + |p_{m-1} - p_{m-2}| + \cdots + |p_{n+1} - p_n| \\ &\leq K^{m-1} |p_1 - p_0| + K^{m-2} |p_1 - p_0| + \cdots + K^n |p_1 - p_0| \\ &= K^n |p_1 - p_0| (1 + K + K^2 + \cdots + K^{m-n-1}) \\ &= K^n |p_1 - p_0| \sum_{j=0}^{m-n-1} K^j \end{aligned}$$

The whole point of this is to now consider this bound as $m \rightarrow \infty$ (which is when

convergence should occur):

$$\begin{aligned}
|p_n - p| &= \lim_{m \rightarrow \infty} |p_n - p_m| \\
&\leq \lim_{m \rightarrow \infty} K^n |p_1 - p_0| \sum_{j=0}^{m-n-1} K^j \\
&= K^n |p_1 - p_0| \lim_{m \rightarrow \infty} \sum_{j=0}^{m-n-1} K^j \\
&= K^n |p_1 - p_0| \sum_{j=0}^{\infty} K^j \\
&= K^n |p_1 - p_0| \frac{1}{1 - K}
\end{aligned}$$

Hence:

$$|p_n - p| \leq \frac{K^n}{1 - K} |p_1 - p_0|$$

□

Using this corollary, we can find an ϵ -accurate approximation by rearranging the equation and using logarithms:

$$\begin{aligned}
n &\geq \frac{\log \left(\frac{\max\{p_0 - a, b - p_0\}}{\epsilon} \right)}{|\log(K)|} \\
n &\geq \frac{\log \left(\frac{|p_1 - p_0|}{\epsilon(1 - K)} \right)}{|\log(K)|}
\end{aligned}$$

Out of both values of n , choose the smaller and use (both are guaranteed to work).

Example: $g(x) = e^{-x}$ **over** $[\frac{1}{4}, 1]$

We want to prove that:

a. $g \in C^{[a,b]}$

$\rightarrow e^{-x}$ is continuous over $\mathbb{R}$

b. $g : [a, b] \mapsto [a, b]$

$\rightarrow g'(x) = -e^{-x} < 0 \ \forall x$

$\therefore g'(x)$ is strictly decreasing

$\therefore \forall x \in [a, b], g(b) \leq g(x) \leq g(a)$

$$\begin{aligned}
g(a) &= e^{-\frac{1}{4}} \geq \frac{1}{4} \\
g(b) &= e^{-1} \leq 1 \\
\therefore g &: [a, b] \mapsto [a, b]
\end{aligned}$$

$$\text{c. } \exists K \in (0, 1) : |g'(x)| \leq K \quad \forall x \in [a, b]$$

$$\rightarrow |g'(x)| = |-e^{-x}| \leq \max_{x \in [\frac{1}{4}, 1]} |-e^{-x}| = \max_{x \in [\frac{1}{4}, 1]} (e^{-x}) = e^{-\frac{1}{4}} < 1$$

$$\therefore K = e^{-\frac{1}{4}}$$

Hence, there exists a unique fixed point $p \in [\frac{1}{4}, 1]$

Example: FPM

Consider an affine FPM, with: $p_0 = 0$, $p_n = g(p_{n-1})$, $g(x) = \frac{1}{2}x + 2$. Prove that the sequence converges.

We can find:

$$\begin{aligned}
\rightarrow p_0 &= 0 \\
\rightarrow p_1 &= g(p_0) = 2 \\
\rightarrow p_2 &= g(p_1) = 1 + 2 = 3 \\
\rightarrow p_3 &= g(p_2) = 1.5 + 2 = 3.5 \\
\rightarrow p_4 &= g(p_3) = 1.75 + 2 = 3.75 \\
&\vdots
\end{aligned}$$

To prove it converges, define the error:

$$E_n = |p_n - p|$$

We will assume that p is the Fixed Point $\implies g(p) = p$. Hence:

$$\begin{aligned}
E_n &= |p_n - p| \\
&= \left| \left(\frac{1}{2}p_{n-1} + 2 \right) - \left(\frac{1}{2}p + 2 \right) \right| \\
&= \frac{1}{2} |p_{n-1} - p| = \frac{1}{2} E_{n-1} \\
&= \frac{1}{2^2} |p_{n-2} - p| = \frac{1}{2} E_{n-2} \\
&\vdots \\
E_n &= \frac{1}{2^n} |p_0 - p| = \frac{p}{2^n}
\end{aligned}$$

$$\therefore \lim_{n \rightarrow \infty} E_n = \lim_{n \rightarrow \infty} \frac{p}{2^n} = 0$$

Hence, the sequence converges.

2.3 Newton's Method

Used for root-finding, subject to conditions.

Say p is a root for $f(x) = 0$, and p_0 is an initial 'close' guess such that:

1. $f(p) = 0$
2. $|p - p_0|$ is close enough that the following arguments are valid
3. $f'(x) \neq 0$ near p or p_0 or anywhere close by
4. the multiplicity of the root is 1

From Taylor's Theorem, we know that:

$$f(x) = f(p_0) + f'(p_0)(x - p_0) + \frac{f''(p_0)}{2!}(x - p_0)^2 + \dots$$

(assuming $f \in C^2[a, b]$, $p, p_0 \in [a, b]$). Plugging in $x = p$:

$$f(p) = 0 = f(p_0) + f'(p_0)(p - p_0) + \frac{f''(p_0)}{2!} \underbrace{(p - p_0)^2}_{\text{negligible}} + \dots$$

$$\therefore 0 = f(p_0) + f'(p_0)(p - p_0)$$

Rearranging:

$$p = p_0 - \frac{f(p_0)}{f'(p_0)}$$

Hence, given an initial value p_0 , Newton's Method is defined as:

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)}, \quad n \geq 0$$

Theorem. (Converge of Newton's Method) Let $f \in C^2[a, b]$. If the root $p \in [a, b]$ is non-degenerate (ie. $f'(p) \neq 0$), then $\exists \delta > 0$ such that Newton's Method generates a sequence for $n = 1, 2, \dots$ that converges to p for any $p_0 \in [p - \delta, p + \delta]$.

Note: in practice, we don't know that δ to start with - the "Basin of Attraction" (interval $[p - \delta, p + \delta]$) is not known before iteration.

We do one of two things to find the roots:

1. Randomize: take several random values for p_0 , and choose the one that converges the fastest
2. Two-Step: use a slower method (like Bisection) for a few iterations, and use that output as the starting p_0 for Newton's Method

If we can't compute $f'(x)$ explicitly, then we can approximate it using a secant line:

$$f'(p_n) \approx \frac{f(p_n) - f(p_{n-1})}{p_n - p_{n-1}}$$

... which can be used to replace the $\frac{f(p_n)}{f'(p_n)}$ term in Newton's Method

2.4 Secant Method

$$p_{n+1} = p_n - \frac{f(p_n)(p_n - p_{n-1})}{f(p_n) - f(p_{n-1})}$$

where p_0 and p_1 are given.

The secant method's error scales like $E_n = \mathcal{O}(E_{n-1}^\alpha)$, where $\alpha = \frac{1+\sqrt{5}}{2} \approx 1.618$

3 Error Analysis

Definition. Let $p_n \rightarrow p$, and $p_n \neq p \forall n \geq 1$. If there exist two constants $\lambda, \alpha > 0$ such that:

$$\lim_{n \rightarrow \infty} \frac{|p_{n+1} - p|}{|p_n - p|^\alpha} = \lambda$$

then p_n converges to p at a rate of α with an asymptotic constant λ .

Remark: the definition asks for α and λ such that:

$$\frac{E_{n+1}}{E_n^\alpha} \rightarrow \lambda < \infty$$

So, for a large enough n (ie. $n \geq N$), $E_{n+1} \sim \lambda E_n^\alpha$

Definition. If $p_n = p \forall n \geq N$, then the sequence converges in *finite time* or *finite steps*. We refer to this as an “ N -step sequence”.

Special Cases

1. if $\alpha = 1$ and $\lambda \in (0, 1)$, then the sequence is called linearly convergent. The error will scale like $E_{n+1} \sim \lambda E_n$ for large n .
2. if $\alpha = 2$, $\lambda > 0$, then the sequence converges quadratically - for example, Newton's Method.
3. if $\lim_{n \rightarrow \infty} \frac{|p_{n+1} - p|}{|p_n - p|} = 0$, then it converges superlinearly. The idea is that $\frac{E_{n+1}}{E_n}$ accelerates to 0 as $n \rightarrow \infty$.

→ if $\exists \alpha > 1 \implies$ Superlinear:

$$\frac{|p_{n+1} - p|}{|p_n - p|} = \frac{|p_{n+1} - p|}{|p_n - p|^\alpha} \cdot |p_n - p|^{\alpha-1}$$

Since $\alpha > 1$, we know that $\alpha - 1 > 0$, and we've assumed $p_n \rightarrow p$. Thus, $|p_n - p|^{\alpha-1} \rightarrow 0$ as $n \rightarrow \infty$

4. if $\lim_{n \rightarrow \infty} \frac{|p_{n+1} - p|}{|p_n - p|} = 1$, it is known as sublinear.

→ the error ratio slows down: $\frac{E_{n+1}}{E_n} \rightarrow 1$, as $n \rightarrow \infty$

→ consider the example: $p_n = \frac{1}{n}$: we know that $p_n \rightarrow p = 0$ as $n \rightarrow \infty$

$$\frac{E_{n+1}}{E_n} = \frac{|p_{n+1} - p|}{|p_n - p|} = \frac{|\frac{1}{n+1} - 0|}{|\frac{1}{n} - 0|} = \frac{n}{n+1} < 1$$

Hence, as $n \rightarrow \infty$, the error ratio $\rightarrow 1$: Sublinear.

Examples

1. $p_n = \frac{1}{2^n}$

$$\rightarrow \frac{E_{n+1}}{E_n} = \frac{\left| \frac{1}{2^{n+1}} - 0 \right|}{\left| \frac{1}{2^n} - 0 \right|} = \frac{2^n}{2^{n+1}} = \frac{1}{2} < 1$$

Hence, the limit as $n \rightarrow \infty$ equals $\frac{1}{2} < 1 \implies \alpha = 1$ and $\lambda = \frac{1}{2}$, so it converges *linearly*.

2. $p_n = \frac{1}{2^{2^n}}$

$$\rightarrow \frac{E_{n+1}}{E_n} = \frac{\left| \frac{1}{2^{2^{n+1}}} - 0 \right|}{\left| \frac{1}{2^{2^n}} - 0 \right|} = \frac{2^{2^n}}{2^{2^{n+1}-2^n}} = \frac{1}{2^{2^{n+1}-2^n}} = \frac{1}{2^{2^n(2-1)}} = \frac{1}{2^{2^n}} \rightarrow 0$$

Hence, it converges *superlinearly*.

$$\rightarrow \text{Now, check for } \alpha = 2: \frac{E_{n+1}}{E_n^2} = \frac{\left| \frac{1}{2^{2^{n+1}}} - 0 \right|}{\left| \frac{1}{2^{2^n}} - 0 \right|^2} = \frac{(2^{2^n})^2}{2^{2^{n+1}}} = \frac{2^{2^n \cdot 2}}{2^{2^{n+1}}} = 1$$

$\therefore$ Quadratically as well.

3. $p_n = \frac{1}{2^{n^2}}$

$$\rightarrow \frac{E_{n+1}}{E_n} = \frac{\left| \frac{1}{2^{(n+1)^2}} - 0 \right|}{\left| \frac{1}{2^{n^2}} - 0 \right|} = \frac{2^{n^2}}{2^{(n+1)^2}} = \frac{1}{2^{2n+1}} \rightarrow 0$$

Hence, it converges *superlinearly*.

$\rightarrow$ Does there $\exists \alpha$ such that $\frac{E_{n+1}}{E_n^\alpha}$ converges?

$$\frac{E_{n+1}}{E_n^\alpha} = \frac{1}{2^{(n+1)^2 - \alpha n^2}} = \frac{1}{2^{(1-\alpha)n^2 + 2n + 1}}. \text{ But } \alpha > 1, \text{ so the fraction } \rightarrow \infty.$$

Hence, it is only superlinear.

4. $p_{n+1} = \frac{p_n}{2} + \frac{1}{p_n}$

→ Fact: $p_n \rightarrow p = \sqrt{2}$ (Newton's Method for $x^2 - 2$)

$$\begin{aligned}
\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n} &= \lim_{n \rightarrow \infty} \frac{|p_{n+1} - \sqrt{2}|}{|p_n - \sqrt{2}|} \\
&= \lim_{n \rightarrow \infty} \frac{\left| \frac{p_n}{2} + \frac{1}{p_n} - \sqrt{2} \right|}{|p_n - \sqrt{2}|} \\
&= \lim_{n \rightarrow \infty} \frac{\left| \frac{p_n^2}{2} - \sqrt{2}p_n + 1 \right|}{|p_n^2 - \sqrt{2}p_n|} \\
&= \lim_{x \rightarrow \sqrt{2}} \frac{\left| \frac{x^2}{2} - \sqrt{2}x + 1 \right|}{|x^2 - \sqrt{2}x|}
\end{aligned}$$

This tends to $\frac{0}{0} \implies$ use L'Hopital's Rule. (since the abs() function is continuous, we can switch the limit and abs()):

$$\begin{aligned}
\lim_{x \rightarrow \sqrt{2}} \frac{\left| \frac{x^2}{2} - \sqrt{2}x + 1 \right|}{|x^2 - \sqrt{2}x|} &= \left| \lim_{x \rightarrow \sqrt{2}} \frac{x - \sqrt{2}}{2x - \sqrt{2}} \right| \\
&= |0| = 0
\end{aligned}$$

Hence, this converges superlinearly.

→ Now, test for quadratic convergence:

$$\begin{aligned}
\lim_{n \rightarrow \infty} \frac{E_{n+1}}{E_n^2} &= \lim_{n \rightarrow \infty} \frac{|p_{n+1} - \sqrt{2}|}{|p_n - \sqrt{2}|^2} \\
&= \lim_{n \rightarrow \infty} \frac{\left| \frac{p_n^2}{2} - \sqrt{2}p_n + 1 \right|}{|p_n| |p_n^2 - 2\sqrt{2}p_n + 2|} \\
&= \lim_{x \rightarrow \sqrt{2}} \frac{\left| \frac{x^2}{2} - \sqrt{2}x + 1 \right|}{|x(x^2 - 2\sqrt{2}x + 2)|} \\
&= \lim_{x \rightarrow \sqrt{2}} \frac{\left| \frac{x^2}{2} - \sqrt{2}x + 1 \right|}{|x^3 - 2\sqrt{2}x^2 + 2x|} = \frac{0}{0} \\
&= \left| \lim_{x \rightarrow \sqrt{2}} \frac{x - \sqrt{2}}{3x^2 - 4\sqrt{2}x + 2} \right| = \frac{0}{0} \\
&= \left| \lim_{x \rightarrow \sqrt{2}} \frac{1}{6x - 4\sqrt{2}} \right| \\
&= \frac{1}{2\sqrt{2}} = \lambda < 1
\end{aligned}$$

Hence, it converges Quadratically as well.

Theorem. Let $g \in C^1[a, b]$, $g : [a, b] \mapsto [a, b]$ and $\exists K \in (0, 1)$ such that $|g'(x)| \leq K \forall x \in [a, b]$. If p is the unique fixed point and $f'(p) \neq 0$, then $\forall p_0 \neq p, p_0 \in [a, b]$, the fixed point converges only linearly to p .

Proof. By assumption, it is guaranteed that $p_n \rightarrow p$, and p is the unique FP. Since $g \in C^1[a, b]$, by the Mean Value Theorem, $\exists \xi_n \in [a, b]$ such that:

$$\begin{aligned} \frac{g(p_n) - p}{p_n - p} &= g'(\xi_n) \\ \therefore g(p_n) - g(p) &= g'(\xi_n) \cdot (p_n - p) \end{aligned}$$

By definition of the FPM and FP:

$$\begin{aligned} \implies p_{n+1} - p &= g'(\xi_n) \cdot (p_n - p) \\ \therefore \frac{|p_{n+1} - p|}{|p_n - p|} &= |g'(\xi_n)| \\ \therefore \lim_{n \rightarrow \infty} \frac{|p_{n+1} - p|}{|p_n - p|} &= \lim_{n \rightarrow \infty} |g'(\xi_n)| \\ &= \left| g' \left(\lim_{n \rightarrow \infty} (\xi_n) \right) \right| \end{aligned}$$

For every n , ξ_n is between p_n and p (by definition), so since the method converges $p_n \rightarrow p$, then $\xi_n \rightarrow p$.

$$\therefore \lim_{n \rightarrow \infty} \frac{|p_{n+1} - p|}{|p_n - p|} = |g'(p)|$$

We know that $g'(p) \neq 0$, so it can't converge faster than linear. We also know that $|g'(x)|$ is bounded above by $K \in (0, 1)$, so it converges linearly. □

Theorem. If $g'(p) = 0$, and g'' is continuous, and $\exists$ an interval around p such that $|g'(x)| \leq M \forall x$ in that interval, then $\exists \delta > 0$ such that $\forall p_0 \in [p - \delta, p + \delta]$ the FPM $p_{n+1} = g(p_n)$ converges quadratically at least.

4 Interpolation

4.1 Lagrange Polynomials

Aim: given $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, we want to find a polynomial that fits all the data points ('interpolates' the $n + 1$ points).

Definition. The points $\{x_i\}_{i=0}^n$ are called the **nodes** (not necessarily ordered)

To be an *interpolant* (interpolating polynomial), $P(x)$ must satisfy two conditions:

i) Interpolation condition: $P(x_i) = f(x_i) (= y_i) \forall x_i, i = 0, 1, \dots, n$

→ this means that each x_i is distinct - don't want a situation where (x, y_1) and (x, y_2) both need to be interpolated, as this would result in a non-function

ii) the degree if $P_n(x) \leq n$

Consider the Linear case: we want to find a degree-1 polynomial that fits $(x_0, f(x_0))$ and $(x_1, f(x_1))$.

$$P_1(x) = f(x_0) \left(\frac{x - x_1}{x_0 - x_1} \right) + f(x_1) \left(\frac{x - x_0}{x_1 - x_0} \right)$$

→ x_0 and x_1 are both constants $\implies$ the function is linear

→ plugging in x_0 and x_1 into $P_1(x)$ gives back $f(x_0)$ and $f(x_1)$ respectively, meaning they are interpolated

The key idea is to construct a polynomial with multiple terms (pre-expansion) that will all cancel to 0 except the one term with the coefficient we want. The big fractions in the brackets above should cancel out, such that:

$$L_{n,k}(x) = \begin{cases} 0 & \text{if } i \neq k \\ 1 & \text{if } i = k \end{cases}$$

$L_{n,k}$ is the **Lagrange Polynomial** of degree n for node k . It is defined as:

$$L_{n,k}(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{k-1})(x - x_{k+1}) \cdots (x - x_n)}{(x_k - x_0)(x_k - x_1) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

$$\therefore L_{n,k}(x) = \prod_{i=0, i \neq k}^n \left(\frac{x - x_i}{x_k - x_i} \right)$$

Theorem. If $\{x_i\}_{i=0}^n$ are $(n+1)$ distinct nodes, and f has finite values at all nodes, then $\exists$ a unique (interpolating) polynomial $P_n(x)$ of degree $\leq n$ such that $f(x_i) = P_n(x_i) \forall i = 0, 1, \dots, n$. Here, $P_n(x)$ is defined as:

$$P_n(x) = f(x_0)L_{n,0}(x) + f(x_1)L_{n,1}(x) + \cdots + f(x_n)L_{n,n}(x)$$

$$\therefore P_n(x) = \sum_{k=0}^n f(x_k)L_{n,k}(x)$$

This ensures that at the node values, the non-essential $L(x)$ cancel out, and leave us with the coefficient $f(x)$ of the correct node, thus interpolating all nodes uniquely.

4.1.1 Error Bounds for Lagrange Polynomials

Theorem. Suppose $x_0, x_1, \dots, x_n$ are $(n+1)$ distinct nodes in an interval $[a, b]$ ($a = \min x_j$, $b = \max x_j$, $f \in C^{n+1}[a, b]$). Then, $\exists \xi(x)$ between $x_0, x_1, \dots, x_n$ (hence, in (a, b)) such that:

$$f(x) = P_n(x) + \underbrace{\frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{j=0}^n (x - x_j)}_{E_n(x)}$$

Aim: to find a suitable step size h such that our function is ϵ -accurate within a given interval (linear interpolation!).

We have a function $f(x)$ on $[a, b]$, and j that satisfies $x_j \leq x \leq x_{j+1}$. Since we are formulating a linear interpolation, $n = 1$, so the error is:

$$|f(x) - P_1(x)| = \left| \frac{f^{(2)}(\xi)}{2} (x - x_j)(x - x_{j+1}) \right| = \frac{|f^{(2)}(\xi)|}{2} |x - x_j| |x - x_{j+1}|$$

Assume the step size is h , so $x_j = jh$ and $x_{j+1} = (j+1)h$:

$$|f(x) - P(x)| \leq \frac{|f^{(2)}(\xi)|}{2} |(x - jh)(x - (j+1)h)|$$

Hence:

$$\begin{aligned} |f(x) - P(x)| &\leq \frac{\max_{\xi \in [a, b]} |f^{(2)}(\xi)|}{2} \max_{x_j \leq x \leq x_{j+1}} |(x - jh)(x - (j+1)h)| \\ &\leq \frac{M}{2} \max_{x_j \leq x \leq x_{j+1}} |(x - jh)(x - (j+1)h)| \end{aligned}$$

Using some hand-wavy bullshit that I don't feel like typing, the max occurs at the critical point when $x = \frac{h^2}{4}$:

$$\begin{aligned} |f(x) - P(x)| &\leq \frac{M}{2} \max_{x_j \leq x \leq x_{j+1}} |g(x)| \\ &\leq \frac{M}{2} \cdot \frac{h^2}{4} \\ \therefore \text{error} &\leq \frac{Mh^2}{8} \leq \epsilon \end{aligned}$$

Hence:

$$h \leq \sqrt{\frac{8\epsilon}{M}}$$

4.2 Data Approximation and Neville's Method

With Lagrange Interpolation, it is guaranteed that the function formed passes through the nodes points. However, this does not tell us anything about the non-nodal points, for which we may want to make estimates. Interpolation pretty much always leads to 'overfitting', so predictions for points are dogshit mostly. To account for this, use Neville's Method, which combines polynomials formed for different degrees.

Definition. Let f be a function defined at points $x_0, x_1, \dots, x_n$, and suppose that $m_1, m_2, \dots, m_k$ are k distinct integers, with $0 \leq m_i \leq n$ for each i . The Lagrange Polynomial that agrees with $f(x)$ at the k points $x_{m_1}, x_{m_2}, \dots, x_{m_k}$ is denoted by $P_{m_1, m_2, \dots, m_k}(x)$.

Let f be defined at the $k + 1$ distinct nodes $x_0, x_1, \dots, x_k$, and let x_j and x_i be two nodes in this set. Then:

$$P_{0,1,\dots,k}(x) = \frac{(x - x_j)P_{0,1,\dots,j-1,j+1,\dots,k}(x) - (x - x_i)P_{0,1,\dots,i-1,i+1,\dots,k}(x)}{x_i - x_j}$$

ie. we can use interpolated polynomials of a lower degree and combine them to form a higher-degree interpolant.

x_0	$P_0 = Q_{0,0}$				
x_1	$P_1 = Q_{1,0}$	$P_{0,1} = Q_{1,1}$			
x_2	$P_2 = Q_{2,0}$	$P_{1,2} = Q_{2,1}$	$P_{0,1,2} = Q_{2,2}$		
x_3	$P_3 = Q_{3,0}$	$P_{2,3} = Q_{3,1}$	$P_{1,2,3} = Q_{3,2}$	$P_{0,1,2,3} = Q_{3,3}$	
x_4	$P_4 = Q_{4,0}$	$P_{3,4} = Q_{4,1}$	$P_{2,3,4} = Q_{4,2}$	$P_{1,2,3,4} = Q_{4,3}$	$P_{0,1,2,3,4} = Q_{4,4}$

As seen, successive combinations can lead to higher degree polynomials at a point.

4.3 Divided Differences

The divided differences method is used to generate coefficients for polynomials.

Let $P_n(x)$ be the n^{th} degree Lagrange Polynomial that agrees with a function f at points $x_0, x_1, \dots, x_n$. This polynomial is unique, but can be represented in different ways that may be useful.

The divided differences of f with respect to $x_0, x_1, \dots, x_n$ are used to express $P_n(x)$ in the form:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_n(x - x_0) \cdots (x - x_{n-1})$$

To determine a_0 , we can simply use $a_0 = P_n(x_0) = f(x_0)$.

When $P(x)$ is evaluated at x_1 , we can find:

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

Now, we can use divided difference notation. The *zeroth divided difference* of the function f with respect to x_i , denoted $f[x_i]$ is simply the value at x_i :

$$f[x_i] = f(x_i)$$

The next divided differences are calculated recursively. The *first divided difference* of f with respect to x_i and x_{i+1} is denoted $f[x_i, x_{i+1}]$ and defined as:

$$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$

The *second divided difference* $f[x_i, x_{i+1}, x_{i+2}]$ is defined as:

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}$$

Similarly, after $k-1$ divided differences have been found, we have $f[x_i, x_{i+1}, \dots, x_{i+k-1}]$ and $f[x_{i+1}, x_{i+2}, \dots, x_{i+k}]$. Hence, the k^{th} divided difference relative to $x_i, x_{i+1}, \dots, x_{i+k}$ is:

$$f[x_i, x_{i+1}, \dots, x_{i+k-1}, x_{i+k}] = \frac{f[x_{i+1}, x_{i+2}, \dots, x_{i+k}] - f[x_i, x_{i+1}, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

This ends with a single n^{th} divided difference:

$$f[x_0, x_1, \dots, x_n] = \frac{f[x_1, x_2, \dots, x_n] - f[x_0, x_1, \dots, x_{n-1}]}{x_n - x_0}$$

Hence, the interpolating polynomial can be rewritten as :

$$P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, \dots, x_k](x - x_0) \cdots (x - x_{k-1})$$

x	$f(x)$	First divided differences	Second divided differences	Third divided differences
x_0	$f[x_0]$			
		$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$	
		$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$		$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$	
		$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2}$		$f[x_1, x_2, x_3, x_4] = \frac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}$
x_3	$f[x_3]$		$f[x_2, x_3, x_4] = \frac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2}$	
		$f[x_3, x_4] = \frac{f[x_4] - f[x_3]}{x_4 - x_3}$		$f[x_2, x_3, x_4, x_5] = \frac{f[x_3, x_4, x_5] - f[x_2, x_3, x_4]}{x_5 - x_2}$
x_4	$f[x_4]$		$f[x_3, x_4, x_5] = \frac{f[x_4, x_5] - f[x_3, x_4]}{x_5 - x_3}$	
		$f[x_4, x_5] = \frac{f[x_5] - f[x_4]}{x_5 - x_4}$		
x_5	$f[x_5]$			

5 Numerical Differentiation

The idea of this section is to use the Lagrange Interpolating form to approximate a “smooth” function by a polynomial, and develop calculus on that interpolant.

Let’s start off with a linear approximation to $f(x)$.

Assume $f \in C^2[a, b]$. Set $x_0, x_1 = x_0 + h$, $h > 0$.

By Lagrange’s Error Formula: if $f \in C^2[x_0, x_1] \implies \exists \xi(x) \in [x_0, x_1]$ such that:

$$f(x) = P_{0,1}(x) + \frac{f''(\xi(x))}{2}(x - x_0)(x - x_1)$$

By Lagrange:

$$P_{0,1}(x) = f(x_0) \frac{(x - x_1)}{(x_0 - x_1)} + f(x_1) \frac{(x - x_0)}{(x_1 - x_0)}$$

Hence, to find $\frac{d}{dx}f(x)$, we use an approximation from $\frac{d}{dx}P_{0,1}(x)$, with an error of $\frac{d}{dx} \left(\frac{f''(\xi(x))}{2}(x - x_0)(x - x_1) \right)$.

First, let’s find $\frac{d}{dx}P_{0,1}(x)$:

$$\begin{aligned} \frac{d}{dx}P_{0,1}(x) &= \frac{d}{dx} \left(f(x_0) \frac{(x - x_1)}{(x_0 - x_1)} + f(x_1) \frac{(x - x_0)}{(x_1 - x_0)} \right) \\ &= f(x_0) \cdot \frac{d}{dx} \left(\frac{x - x_1}{x_0 - x_1} \right) + f(x_1) \cdot \frac{d}{dx} \left(\frac{x - x_0}{x_1 - x_0} \right) \\ &= f(x_0) \frac{1}{x_0 - x_1} + f(x_1) \frac{1}{x_1 - x_0} \\ &= \frac{f(x_0)}{-h} + \frac{f(x_1)}{h} \\ &= \frac{f(x_1) - f(x_0)}{h} \\ &= \frac{f(x_0 + h) - f(x_0)}{h} \end{aligned}$$

Now, for the error term:

$$\begin{aligned} \frac{d}{dx}\text{error}(x) &= \frac{d}{dx} \left(\frac{f''(\xi(x))}{2}(x - x_0)(x - x_1) \right) \\ &= \frac{(x - x_1) + (x - x_0)}{2} f''(\xi(x)) + \frac{(x - x_0)(x - x_1)}{2} \frac{d}{dx} (f''(\xi(x))) \end{aligned}$$

However, we only know that $f \in C^2[x_0, x_1]$, so finding the third derivative of f could be problematic.

Thus, we substitute x for one of the nodes (set $x = x_0$), and this causes the second term to cancel out.

$$\begin{aligned}\frac{d}{dx}\text{error}(x_0) &= \frac{(x_0 - x_1) + (x_0 - x_0)}{2} f''(\xi(x_0)) \\ &= \frac{x_0 - x_1}{2} f''(\xi_0) \\ &= \frac{-h}{2} f''(\xi_0)\end{aligned}$$

Finally, this gives us:

$$\frac{d}{dx}f(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(\xi_0)$$

where $\xi_0 \in [x_0, x_0 + h]$

If we had set the interval to be $[x_0 - h, x_0]$, by symmetry, we'd get:

$$\frac{d}{dx}f(x_0) = \frac{f(x_0) - f(x_0 - h)}{h} + \frac{h}{2} f''(\xi_0)$$

where $\xi_0 \in [x_0, x_0 + h]$

In general, this can be summarized to:

$$f(x) = \sum_{j=0}^n f(x_j) L_{j,n}(x) + \frac{(x - x_0) \cdots (x - x_n)}{(n + 1)!} f^{(n+1)}(\xi(x))$$

and:

$$\frac{d}{dx}f(x) = \sum_{j=0}^n f(x_j) \cdot \frac{d}{dx}L_{j,n}(x) + \frac{d}{dx}E_L(x)$$

Once we set x to one of the nodes x_k :

$$\frac{d}{dx}f(x_k) = \sum_{j=0}^n \left(f(x_j) \frac{d}{dx}L_{j,n}(x) \Big|_{x=x_k} \right) + \frac{d}{dx}E_L(x) \Big|_{x=x_k}$$

All in all:

$$f'(x_k) = \sum_{j=0}^n f(x_j) L'_{j,n}(x_k) + \frac{f^{(n+1)}(\xi(x_k))}{(n + 1)!} \prod_{j=0, j \neq k}^n (x_k - x_j)$$

Examples

→ Forward difference, 2nd order

The nodes are $x_0, x_0 + h, x_0 + 2h$:

$$\frac{d}{dx}f(x_0) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + \frac{h^2}{3}f^{(3)}(\xi_0)$$

where $\xi_0 \in [x_0, x_0 + 2h]$

→ Backward difference, 2nd order

The nodes are $x_0, x_0 - h, x_0 - 2h$:

$$\frac{d}{dx}f(x_0) = \frac{3f(x_0) - 4f(x_0 - h) + f(x_0 - 2h)}{2h} + \frac{h^2}{3}f^{(3)}(\xi_1)$$

where $\xi_1 \in [x_0 - 2h, x_0]$

→ Centered, 2nd order

The nodes are $x_0 - h, x_0, x_0 + h$:

$$\frac{d}{dx}f(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \frac{h^2}{6}f^{(3)}(\xi_2)$$

where $\xi_2 \in [x_0 - h, x_0 + h]$

Let's try this in an example: find the approximation for $f(x)$ based on the central difference, 2nd order.

Central and 2nd order means that the nodes are: $x_0, x_1 = x_0 - h$ and $x_2 = x_0 + h$.

$$\begin{aligned} \text{i. } L_0(x) &= \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} = \frac{(x - x_0 + h)(x - x_0 - h)}{-h^2} \\ \therefore \frac{d}{dx}L_0(x) \Big|_{x=x_0} &= \left(\frac{x - x_0 - h}{-h^2} + \frac{x - x_0 + h}{-h^2} \right) \Big|_{x=x_0} \\ &= \frac{-h}{-h^2} + \frac{h}{-h^2} \\ &= 0 \end{aligned}$$

Thus, $f(x_0)$ won't feature in the central difference.

$$\begin{aligned} \text{ii. } L_1(x) &= \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} = \frac{(x - x_0)(x - x_0 - h)}{2h^2} \\ \therefore \frac{d}{dx}L_1(x) \Big|_{x=x_0} &= \left(\frac{x - x_0 - h}{2h^2} + \frac{x - x_0}{2h^2} \right) \Big|_{x=x_0} \\ &= \frac{-h}{2h^2} + 0 \\ &= \frac{-1}{2h} \end{aligned}$$

$$\begin{aligned}
\text{iii. } L_2(x) &= \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)} = \frac{(x-x_0)(x-x_0+h)}{2h^2} \\
\therefore \frac{d}{dx}L_2(x) \Big|_{x=x_0} &= \left(\frac{x-x_0+h}{2h^2} + \frac{x-x_0}{2h^2} \right) \Big|_{x=x_0} \\
&= \frac{h}{2h^2} + 0 \\
&= \frac{1}{2h}
\end{aligned}$$

Combining these calculated functions for L' , we can combine them:

$$\begin{aligned}
\sum_{j=0}^2 f(x_j)L'_{j,2}(x_0) &= f(x_0) \cdot 0 + f(x_1) \cdot \frac{-1}{2h} + f(x_2) \frac{1}{2h} \\
&= \frac{-1}{2h}f(x_0-h) + \frac{1}{2h}f(x_0+h) \\
&= \frac{f(x_0+h) - f(x_0-h)}{2h}
\end{aligned}$$

The error term is:

$$\begin{aligned}
\frac{f^{(n+1)}(\xi(x_k))}{(n+1)!} \prod_{j=0, j \neq k}^n (x_k - x_j) &= \frac{f^{(3)}(\xi(x_0))}{3!} (x_0 - x_1)(x_0 - x_2) \\
&= \frac{-h^2}{6} f^{(3)}(\xi_0) \\
&= \mathcal{O}(h^2)
\end{aligned}$$

Hence:

$$f'(x_0) = \frac{f(x_0+h) - f(x_0-h)}{2h} + \mathcal{O}(h^2)$$

for $f \in C^3[x_0-h, x_0+h]$

5.1 Method of Undetermined Coefficients

There's not much I can explain in theory. Just use this example to understand I guess. I'm slowly losing my mind as the quarter progresses. To the person reading these notes: I hope you have a good day.

Problem: find an approximation to $f^{(4)}(x)$ using 5 nodes, with an error of $\mathcal{O}(h^2)$

Solution: first of all, we know that the fourth derivative of f will have terms in the order of $\frac{1}{h^4}$. Hence, to obtain an error term of $\mathcal{O}(h^2)$, we need to expand using Taylor's theorem till the sixth power.

Let the 5 nodes be $x_0 + 2h$, $x_0 + h$, x_0 , $x_0 - h$ and $x_0 - 2h$, where $h > 0$:

$$f^{(4)}(x_0) = Af(x_0 + 2h) + Bf(x_0 + h) + Cf(x_0) + Df(x_0 - h) + Ef(x_0 - 2h) + \mathcal{O}(h^2)$$

Using Taylor's Theorem for the terms on the RHS:

$$\begin{aligned} \rightarrow f(x_0 \pm h) &= f(x_0) \pm hf^{(1)}(x_0) + \frac{h^2}{2}f^{(2)}(x_0) \pm \frac{h^3}{6}f^{(3)}(x_0) \\ &\quad + \frac{h^4}{4!}f^{(4)}(x_0) \pm \frac{h^5}{5!}f^{(5)}(x_0) + \frac{h^6}{6!}f^{(6)}(\xi_{\pm 1}) \end{aligned}$$

where $\xi_{+1} \in [x_0, x_0 + h]$ and $\xi_{-1} \in [x_0 - h, x_0]$

$$\begin{aligned} \rightarrow f(x_0 \pm 2h) &= f(x_0) \pm 2hf^{(1)}(x_0) + \frac{(2h)^2}{2}f^{(2)}(x_0) \pm \frac{(2h)^3}{6}f^{(3)}(x_0) \\ &\quad + \frac{(2h)^4}{4!}f^{(4)}(x_0) \pm \frac{(2h)^5}{5!}f^{(5)}(x_0) + \frac{(2h)^6}{6!}f^{(6)}(\xi_{\pm 2}) \end{aligned}$$

where $\xi_{+2} \in [x_0, x_0 + 2h]$ and $\xi_{-2} \in [x_0 - 2h, x_0]$

This is too long to write in expanded form with the coefficients, so let's order it. When expanded using Taylor's Theorem, the representation for $f^{(4)}(x_0)$ contains the following terms:

- $(A + B + C + D + E)f(x_0)$
- $(2A + B - D - 2E)hf^{(1)}(x_0)$
- $(4A + B + D + 4E)\frac{h^2}{2}f^{(2)}(x_0)$
- $(8A + B - D - 8E)\frac{h^3}{3!}f^{(3)}(x_0)$
- $(16A + B + D + 16E)\frac{h^4}{4!}f^{(4)}(x_0)$
- $(32A + B - D - 32E)\frac{h^5}{5!}f^{(5)}(x_0)$
- $\frac{64Ah^6}{6!}f^{(6)}(\xi_{+2}) + \frac{Bh^6}{6!}f^{(6)}(\xi_{+1}) - \frac{Dh^6}{6!}f^{(6)}(\xi_{-1}) - \frac{64Eh^6}{6!}f^{(6)}(\xi_{-2})$

Since there are 5 unknowns, we can use the first five equations (corresponding to $f(x_0)$ through $f^{(4)}(x_0)$) to calculate their values.

Hence, the five equations we get are:

$$\begin{cases} A + B + C + D + E = 0 \\ 2A + B - D - 2E = 0 \\ 4A + B + D + 4E = 0 \\ 8A + B - D - 8E = 0 \\ (16A + B + D + 16E) \frac{h^4}{4!} = 1 \end{cases}$$

Once we rearrange the last equation to $16A + B + D + 16E = \frac{24}{h^4}$, we can solve this computationally to get:

$$\begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \frac{1}{h^4} \begin{bmatrix} 1 \\ -4 \\ 6 \\ -4 \\ 1 \end{bmatrix}$$

Additionally:

$$\begin{aligned} \text{error term} &= \frac{64Ah^6}{6!}f^{(6)}(\xi_{+2}) + \frac{Bh^6}{6!}f^{(6)}(\xi_{+1}) - \frac{Dh^6}{6!}f^{(6)}(\xi_{-1}) - \frac{64Eh^6}{6!}f^{(6)}(\xi_{-2}) \\ &= \mathcal{O}((|A| + |B| + |C| + |D|)h^6) \\ &= \mathcal{O}(h^2) \end{aligned}$$

Hence:

$$f^{(4)}(x_0) = \frac{f(x_0 + 2h) - 4f(x_0 + h) + 6f(x_0) - 4f(x_0 - h) + f(x_0 - 2h)}{h^4} + \mathcal{O}(h^2)$$

5.2 2nd Order Finite Differences

Taylor's Theorem in 2D Suppose $f(x, y)$ and all its partial derivatives of order $\leq (n + 1)$ are continuous on some region $R \in \mathbb{R}^2$. Let $(x_0, y_0) \in R$. Then, for every $(\xi, \mu) \in R$, with ξ between x and x_0 , and μ between y and y_0 , we have:

$$\begin{aligned} f(x, y) &= f(x_0, y_0) + \left((x - x_0) \frac{\partial f}{\partial x}(x_0, y_0) + (y - y_0) \frac{\partial f}{\partial y}(x_0, y_0) \right) \\ &\quad + \left(\frac{(x - x_0)^2}{2} \frac{\partial^2 f}{\partial x^2}(x_0, y_0) + (x - x_0)(y - y_0) \frac{\partial^2 f}{\partial x \partial y}(x_0, y_0) + \frac{(y - y_0)^2}{2} \frac{\partial^2 f}{\partial y^2}(x_0, y_0) \right) \\ &\quad + \dots \end{aligned}$$

Or, if combined to a single formula:

$$f(x, y) = P_n(x) + R_n(x)$$

where:

- $P_n(x) = \frac{1}{n!} \sum_{j=0}^n \binom{n}{j} (x - x_0)^{n-j} (y - y_0)^j \frac{\partial^n f}{\partial x^{n-j} \partial y^j}(x_0, y_0)$
- $R_n(x) = \frac{1}{(n+1)!} \sum_{j=0}^{n+1} \binom{n+1}{j} (x - x_0)^{n+1-j} (y - y_0)^j \frac{\partial^{n+1} f}{\partial x^{n+1-j} \partial y^j}(\xi, \mu)$

If we use an equally spaced grid with spacing $\Delta x > 0$ and $\Delta y > 0$, we can substitute $(x - x_0)$ for Δx and $(y - y_0)$ for Δy in the same formulae.

Examples

→ First Derivative:

$$\frac{\partial f}{\partial x}(x_0, y_0) = \frac{f(x_0 + \Delta x, y_0) - f(x_0, y_0)}{\Delta x} + \mathcal{O}(\Delta x)$$

(can be verified by applying Taylor's Theorem to the RHS, like the method of undetermined coefficients).

The same formula can be used for the first derivative in the y direction: using Δy in the correct places.

→ Second Derivative (in one direction):

$$\frac{\partial^2 f}{\partial x^2}(x_0, y_0) = \frac{f(x_0 + \Delta x, y_0) - 2f(x_0, y_0) + f(x_0 - \Delta x, y_0)}{\Delta x^2} + \mathcal{O}(\Delta x^2)$$

(once again, similar formula can be used for $\frac{\partial^2 f}{\partial y^2}$)

→ Second Derivative (in both directions):

$$\begin{aligned} \frac{\partial^2 f}{\partial x \partial y} &= \frac{1}{4\Delta x \Delta y} (f(x_0 + \Delta x, y_0 + \Delta y) - f(x_0 + \Delta x, y_0 - \Delta y) \\ &\quad - f(x_0 - \Delta x, y_0 + \Delta y) + f(x_0 - \Delta x, y_0 - \Delta y)) \\ &\quad + \mathcal{O}(\max\{\Delta x^2, \Delta y^2\}) \end{aligned}$$

5.3 Imaging - Edge Detection

Let $f(x, y) = I(x, y)$ be the function of light intensity (grayscale image).

To detect objects, we will need to calculate:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x}(x, y) \\ \frac{\partial f}{\partial y}(x, y) \end{bmatrix}$$

$$\begin{aligned}
\therefore |\nabla f(x_0, y_0)|^2 &= \left| \frac{\partial f}{\partial x}(x_0, y_0) \right|^2 + \left| \frac{\partial f}{\partial y}(x_0, y_0) \right|^2 \\
&= \left| \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x} \right|^2 + \left| \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y} \right|^2 \\
&= |f(x_0 + 1, y_0) - f(x_0, y_0)|^2 + |f(x_0, y_0 + 1) - f(x_0, y_0)|^2
\end{aligned}$$

since for an image, $\Delta x = \Delta y = 1$.

We can define a ‘threshold’ $\gamma > 0$ as a 0 vs 1 classifier for different pixels, and process the image through a threshold function like such:

$$T(|\nabla f(x, y)|) = \begin{cases} 1 & \text{if } |\nabla f(x, y)| \geq \gamma \\ 0 & \text{if } |\nabla f(x, y)| < \gamma \end{cases}$$

In this way, we can sort-of detect objects etc. from a given image.

6 Numerical Integration

Aim: to numerically approximate $\int_a^b f(x) dx$.

6.1 Closed Quadrature Rules

Quadrature rules take the form:

$$\int_a^b f(x) dx \approx \sum_{j=0}^n c_j f(x_j)$$

To get the general form, take a set of nodes $\{x_0, x_1, \dots, x_n\} \subseteq [a, b]$, We will approximate $f(x)$ by the interpolant $P_n(x)$ based on the Lagrange form, and then integrate by hand:

$$\begin{aligned} f(x) &= \sum_{j=0}^n f(x_j) L_j(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{j=0}^n (x - x_j) \\ \therefore \int_a^b f(x) dx &= \int_a^b \sum_{j=0}^n f(x_j) L_j(x) dx + \underbrace{\int_a^b \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \prod_{j=0}^n (x - x_j) dx}_{E_I} \\ &= \sum_{j=0}^n \left(f(x_j) \int_a^b L_j(x) dx \right) + E_I \\ &= \sum_{j=0}^n c_j f(x_j) + E_I \end{aligned}$$

where $c_j = \int_a^b L_j(x) dx$

Trapezoidal Rule

Consider two nodes: $x_0 = a$, $x_1 = b$, with spacing $h = b - a > 0$. Our aim is to approximate $\int_a^b f(x) dx$ using degree-1 $\int_a^b P_1(x) dx$.

$$\begin{aligned} P_1(x) &= f(x_0) \left(\frac{x - x_1}{x_0 - x_1} \right) + f(x_1) \left(\frac{x - x_0}{x_1 - x_0} \right) \\ &= f(x_0) \left(\frac{x - x_1}{-h} \right) + f(x_1) \left(\frac{x - x_0}{h} \right) \\ &= f(a) \left(\frac{b - x}{h} \right) + f(b) \left(\frac{x - a}{h} \right) \end{aligned}$$

Now, integrating:

$$\begin{aligned}
\int_a^b P_1(x) \, dx &= f(a) \int_a^b \frac{b-x}{h} \, dx + f(b) \int_a^b \frac{x-a}{h} \, dx \\
&= f(a) \left(\frac{-(b-x)^2}{2h} \Big|_a^b \right) + f(b) \left(\frac{(x-a)^2}{2h} \Big|_a^b \right) \\
&= f(a) \left(0 + \frac{(b-a)^2}{2h} \right) + f(b) \left(\frac{(b-a)^2}{2h} - 0 \right) \\
&= f(a) \frac{h^2}{2h} + f(b) \frac{h^2}{2h} \\
&= \frac{h}{2} (f(a) + f(b))
\end{aligned}$$

Theorem. $\int_a^b g(x)w(x) \, dx = g(\gamma) \int_a^b w(x) \, dx$, for some $\gamma \in [a, b]$

We can use this theorem to find the error in the integration:

$$\begin{aligned}
E_I &= \int_a^b \frac{f^{(2)}(\xi(x))}{2} (x-a)(x-b) \, dx \\
&= -\frac{1}{2} \int_a^b f^{(2)}(\xi(x))(x-a)(b-x) \, dx \\
&= -\frac{1}{2} f^{(2)}(\tilde{\xi}) \int_a^b (x-a)(b-x) \, dx \\
&= -\frac{1}{2} \frac{(b-a)^3}{6} f^{(2)}(\tilde{\xi}) \\
&= -\frac{h^3}{12} f^{(2)}(\tilde{\xi})
\end{aligned}$$

for some $\tilde{\xi} \in [a, b]$

Finally:

$$\int_a^b f(x) \, dx = \frac{h}{2} (f(a) + f(b)) - \frac{h^3}{12} f^{(2)}(\tilde{\xi})$$

Simpson's Rule

For $x_0 = a$, $x_1 = a + h$, $x_2 = a + 2h = b$, where $h = \frac{b-a}{2} > 0$, we use P_2 as an interpolant, giving:

$$\int_a^b f(x) \, dx = \frac{h}{3} (f(x_0) + 4f(x_1) + f(x_2)) - \frac{h^5}{90} f^{(4)}(\tilde{\xi})$$

for some $\tilde{\xi} \in [a, b]$

Exactness: a quadrature rule is exact for a degree- n polynomial if:

$$\int_a^b f(x) \, dx = \sum_{j=0}^n c_j f(x_j)$$

holds $\forall f \in P_n(\mathbb{R})$

Accuracy / Precision: the degree of accuracy or the precision of a quadrature rule is the largest positive integer n such that the rule is exact for x^k for $k = 0, 1, \dots, n$, but not exact for x^{n+1} .

Example

Find weights c_i , $i = 0, 1, 2, 3$ such that:

$$\int_0^1 f(x) \, dx = c_0 f(x_0) + c_1 f(x_1) + c_2 f(x_2) + c_3 f(x_3)$$

where $x_i = ih$, $h = \frac{1}{3}$, has the highest precision possible.

We have 4 unknowns, hence we will need to set up a 4×4 invertible linear system to find all c_i using exactness for degrees upto 3.

→ for $f(x) = 1$:

$$\text{LHS} = \int_0^1 dx = 1$$

$$\text{RHS} = c_0 \cdot 1 + c_1 \cdot 1 + c_2 \cdot 1 + c_3 \cdot 1$$

Hence:

$$c_0 + c_1 + c_2 + c_3 = 1$$

→ for $f(x) = x$:

$$\text{LHS} = \int_0^1 x \, dx = \frac{1}{2}$$

$$\text{RHS} = c_0 \cdot 0 + c_1 \cdot \frac{1}{3} + c_2 \cdot \frac{2}{3} + c_3 \cdot 1$$

Hence:

$$\frac{1}{3}c_1 + \frac{2}{3}c_2 + c_3 = \frac{1}{2}$$

→ for $f(x) = x^2$:

$$\text{LHS} = \int_0^1 x^2 \, dx = \frac{1}{3}$$

$$\text{RHS} = c_0 \cdot 0 + c_1 \cdot \frac{1}{9} + c_2 \cdot \frac{4}{9} + c_3 \cdot 1$$

Hence:

$$\frac{1}{9}c_1 + \frac{4}{9}c_2 + c_3 = \frac{1}{3}$$

→ for $f(x) = x^3$:

$$\text{LHS} = \int_0^1 x^3 dx = \frac{1}{4}$$

$$\text{RHS} = c_0 \cdot 0 + c_1 \cdot \frac{1}{27} + c_2 \cdot \frac{8}{27} + c_3 \cdot 1$$

Hence:

$$\frac{1}{27}c_1 + \frac{8}{27}c_2 + c_3 = \frac{1}{4}$$

This gives the system:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & \frac{1}{3} & \frac{2}{3} & 1 \\ 0 & \frac{1}{9} & \frac{8}{9} & 1 \\ 0 & \frac{1}{27} & \frac{8}{27} & 1 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 1/3 \\ 1/4 \end{bmatrix}$$

Solving this system gives:

$$\begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 1/8 \\ 3/8 \\ 3/8 \\ 1/8 \end{pmatrix}$$

Hence:

$$\int_0^1 f(x) dx = \frac{1}{8}f(0) + \frac{3}{8}f\left(\frac{1}{3}\right) + \frac{3}{8}f\left(\frac{2}{3}\right) + \frac{1}{8}f(1)$$

This is exact for degree 3.

To check the precision - show that the rule fails for some degree x^k , $k \geq 4$.

→ $f(x) = x^4$:

$$\text{LHS} = \int_0^1 x^4 dx = \frac{1}{5}$$

$$\text{RHS} = \frac{1}{8} \cdot 0 + \frac{3}{8} \cdot \frac{1}{81} + \frac{3}{8} \cdot \frac{16}{81} + \frac{3}{8} \cdot 1 = \frac{49}{108} \neq \text{LHS}$$

Hence, the rule fails for $x^4 \implies$ the maximum precision is attained with a cubic polynomial.

The error here scales like: $\mathcal{O}(h^5 f^{(4)}(\xi))$

Simpson's $\frac{3}{8}$ Rule

$$\int_{x_0}^{x_3} f(x) dx = \frac{3h}{8} (f(x_0) + 3f(x_1) + 3f(x_2) + f(x_3)) - \frac{3}{80} h^5 f^{(4)}(\xi)$$

where $\xi \in [x_0, x_3]$

Note: if we have $\int_a^b f(x) dx$, we can map $[a, b] \mapsto [0, 1]$ (ie. $x_0 \mapsto \tau$)

$$\tau = \frac{x - a}{b - a} \implies d\tau = \frac{dx}{b - a}$$

$$\therefore \int_a^b f(x) dx = (b - a) \int_0^1 f((b - a)\tau + a) d\tau$$

General formula for closed quadratures: $x_0 = a$, $x_n = b$, $h = \frac{b - a}{n}$ ie. $(n + 1)$ nodes.

- n is even, $f \in C^{n+2}[a, b]$:

$$\begin{aligned} E(f) &= \frac{h^{n+3} f^{(n+2)}(\xi)}{(n + 2)!} \int_0^n t^2(t - 1) \cdots (t - n) dt \\ &= C_n h^{n+3} f^{(n+2)}(\xi) \\ &= \mathcal{O}(h^{n+3}) \end{aligned}$$

- n is odd, $f \in C^{n+1}[a, b]$:

$$\begin{aligned} E(f) &= \frac{h^{n+2} f^{(n+1)}(\xi)}{(n + 1)!} \int_0^n t(t - 1) \cdots (t - n) dt \\ &= C_n h^{n+2} f^{(n+1)}(\xi) \\ &= \mathcal{O}(h^{n+2}) \end{aligned}$$

Hence, when n is even, there is an extra degree of precision.

6.2 Open Quadrature Rules

In open rules the endpoints are not included. The endpoints are instead “ghost” nodes, $x_{-1} = a$, $x_{n+1} = b$, and the quadrature nodes: $x_0 = a + h$, $x_n = b - h$

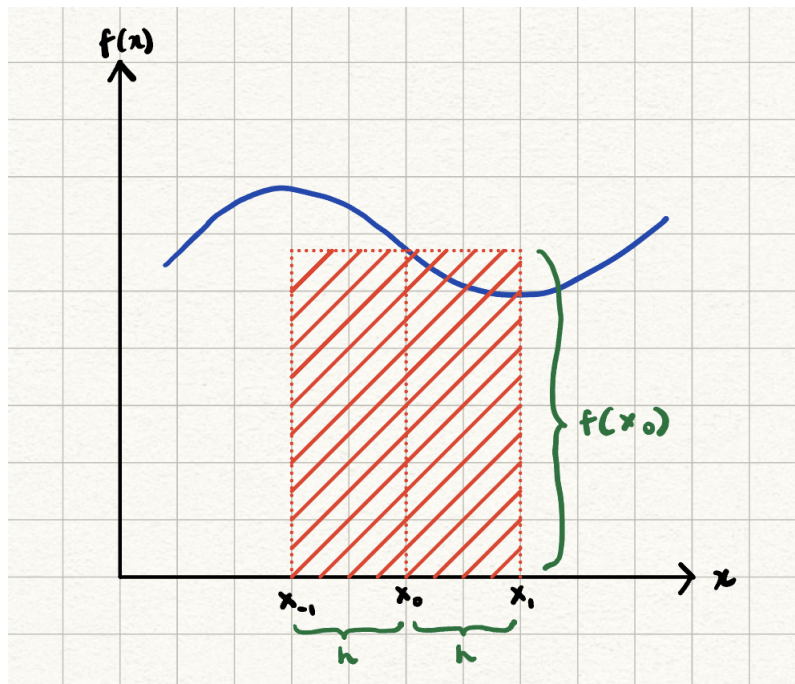
$$\implies h = \frac{b - a}{n + 2}$$

Hence, for $n = 0$:

$$\begin{aligned}\int_a^b f(x) \, dx &= \int_{x_{-1}}^{x_1} f(x) \, dx \\ &= 2hf(x_0) + \frac{h^3}{3}f^{(2)}(\xi)\end{aligned}$$

for $\xi \in [a, b]$

This is the “**Midpoint Rule**” - basically how you approximate the integral using a constant and a single node:



For $n = 1$, we approximate $f(x)$ by a line through the nodes x_0 and x_1 , leading to:

$$\int_a^b f(x) \, dx = \int_{x_{-1}}^{x_2} f(x) \, dx = \frac{3h}{2} (f(x_0) + f(x_1)) + \frac{3h^3}{4}f^{(2)}(\xi)$$

for some $\xi \in [a, b]$.

For $n = 2$, we approximate $f(x)$ by a quadratic through the nodes x_0 , x_1 and x_2 which yields:

$$\int_a^b f(x) \, dx = \int_{x_{-1}}^{x_3} f(x) \, dx = \frac{4h}{3} (2f(x_0) - f(x_1) + 2f(x_2)) + \frac{14h^5}{45}f^{(4)}(\xi)$$

for some $\xi \in [a, b]$.

The error term for an open quadrature rule is similar to the closed quadrature one, but with different limits:

- n is even, $f \in C^{n+2}[a, b]$:

$$\begin{aligned} E(f) &= \frac{h^{n+3} f^{(n+2)}(\xi)}{(n+2)!} \int_{-1}^{n+1} t^2(t-1) \cdots (t-n) dt \\ &= C_n h^{n+3} f^{(n+2)}(\xi) \\ &= \mathcal{O}(h^{n+3}) \end{aligned}$$

- n is odd, $f \in C^{n+1}[a, b]$:

$$\begin{aligned} E(f) &= \frac{h^{n+2} f^{(n+1)}(\xi)}{(n+1)!} \int_{-1}^{n+1} t(t-1) \cdots (t-n) dt \\ &= C_n h^{n+2} f^{(n+1)}(\xi) \\ &= \mathcal{O}(h^{n+2}) \end{aligned}$$

6.3 Composite Quadrature Rules

Quadrature rules are derived from polynomial interpolation of $f(x)$ over $[a, b]$ (the interval of integration).

For higher-order accuracy, the quadrature rules need higher levels of differentiability.

As seen in interpolation, it can be more effective to approximate $f(x)$ by local lower-order polynomials over subintervals (which requires less differentiability than a higher order method over the entire interval).

We will use this idea to construct composite rules, that is, we will break up the interval into subintervals and approximate the integrals on each subinterval.

Method

Consider computing the integral $\int_a^b f(x) dx$. We will subdivide $[a, b]$ into $n > 0$ equally-sized subintervals.

We set the spacing to $h = \frac{b-a}{n}$, and define the nodes:

$$x_j = a + jh$$

for $j = 0, 1, \dots, n$.

Thus, the integral over $[a, b]$ can be written as the sum of integrals over each subinterval:

$$\int_a^b f(x) dx = \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \cdots + \int_{x_{n-1}}^{x_n} f(x) dx$$

Then, we compute each integral $\int_{x_j}^{x_{j+1}} f(x) dx$ using the trapezoidal rule with $h = x_{j+1} - x_j = \frac{b-a}{n}$

Applying the trapezoidal rule to each part:

$$\begin{aligned} \int_a^b f(x) dx &= \int_{x_0}^{x_1} f(x) dx + \int_{x_1}^{x_2} f(x) dx + \cdots + \int_{x_{n-1}}^{x_n} f(x) dx \\ &\approx \frac{h}{2} [f(x_0) + f(x_1)] + \frac{h}{2} [f(x_1) + f(x_2)] + \cdots + \frac{h}{2} [f(x_{n-1}) + f(x_n)] \\ &= \frac{h}{2} \left(f(x_0) + 2 \sum_{j=1}^{n-1} f(x_j) + f(x_n) \right) \\ \therefore \int_a^b f(x) dx &= \frac{h}{2} \left(f(a) + 2 \sum_{j=1}^{n-1} f(x_j) + f(b) \right) \end{aligned}$$

Now, we can derive an error formula by calculating the error for each sub-integral and adding them up: First:

$$\int_a^b f(x) dx = \sum_{j=0}^{n-1} \int_{x_j}^{x_{j+1}} f(x) dx$$

where $[a, b] = [x_0, x_1] \cup [x_1, x_2] \cup \cdots \cup [x_{n-1}, x_n]$

Using the trapezoidal error formula (for each subinterval):

$$\begin{aligned} \int_a^b f(x) dx &= \sum_{j=0}^{n-1} \int_{x_j}^{x_{j+1}} f(x) dx \\ &= \sum_{j=0}^{n-1} \left\{ \frac{h}{2} [f(x_j) + f(x_{j+1})] - \frac{h^3}{12} f^{(2)}(\xi_j) \right\} \\ &= \frac{h}{2} \left[f(x_0) + 2 \sum_{j=1}^{n-1} f(x_j) + f(x_n) \right] - \frac{h^3}{12} f^{(2)}(\xi_j) \end{aligned}$$

for some $\xi_j \in [x_j, x_{j+1}]$. Each subinterval likely has a different ξ_j associated with it.

We've reached the part of the derivation where I stop giving a shit and write the final answer. Define $h_n = \frac{b-a}{n}$ for the Composite Quadrature Rule:

$$\int_a^b f(x) dx = \frac{h_n}{2} \left(f(a) + 2 \sum_{j=1}^{n-1} f(x_j) + f(b) \right) - \frac{h_n^2(b-a)}{12} f^{(2)}(\xi)$$

The error for this scales like $\mathcal{O}\left(\frac{(b-a)^3}{n^2}\right)$

For the trapezoidal rule, take $h_1 = b - a$:

$$\int_a^b f(x) dx = \frac{h_1}{2}(f(x_0) + f(x_1)) - \frac{h_1^3}{12}f^{(2)}(\xi)$$

The error for this scales like $\mathcal{O}((b-a)^3)$

Error

The magnitude of error associated with the composite trapezoidal rule is given by:

$$E(f) = \frac{h^2(b-a)}{12} |f^{(2)}(\xi)| \leq \frac{h^2(b-a)}{12} \max_{x \in [a,b]} |f^{(2)}(x)|$$

where we take an upper bound based on the maximum of the absolute value of the second derivative.

We know $b - a = 2$, so $h^2 = \frac{(b-a)^2}{n^2} = \frac{4}{n^2}$

The number of nodes to maintain an error threshold of ϵ needed is: $n \geq \sqrt{\frac{M(b-a)^3}{12\epsilon}}$
(here, $M = \max_{x \in [a,b]} |f^{(2)}(x)|$)

6.4 Composite Midpoint Rule

For $f \in C^2[a, b]$, n is even, $h = \frac{b-a}{n+2}$, $x_j = a + (j+1)h$, $j = -1, 0, \dots, n, n+1$, with two ghost nodes, then there exists a $\xi \in (a, b)$ such that:

$$\int_a^b f(x) dx = 2h \sum_{j=0}^{n/2} f(x_{2j}) + \frac{h^2(b-a)}{6} f^{(2)}(\xi)$$

In each subinterval $[x_{2j-1}, x_{2j+1}]$, we estimate the function $f(x)$ by the constant value $f(x_{2j})$ i.e. the midpoint. This is a piece-wise constant approximation to $f(x)$, which leads to the approximation:

$$\begin{aligned} \int_a^b f(x) dx &\approx 2h \sum_{j=0}^{n/2} f(x_{2j}) \\ &= 2h(f(x_0) + f(x_2) + \dots + f(x_n)) \\ &= 2h(f(a+h) + f(a+3h) + \dots + f(a+(n-1)h)) \end{aligned}$$

To determine n so that the error is within a certain threshold ϵ ie. $|E(f)| \leq \epsilon$, when $|f''(x)| \leq M$, we can take the upper bound:

$$|E(f)| \leq \frac{Mh^2(b-a)}{6} = \frac{M(b-a)^3}{6(n+2)^2} \leq \epsilon$$

$$\therefore n \geq \sqrt{\frac{M(b-a)^3}{6\epsilon}} - 2$$

6.5 Gaussian Quadrature

Till now, we were deriving Quadrature Rules using Lagrange Interpolation ($f(x) \approx P_n(x)$, then integrate both), or by the method of undetermined coefficients with equally spaced nodes (both open and closed rules). So we have a spacing of $h > 0$ which is uniform/fixed/equally-spaced, and if we're using $(n + 1)$ nodes for $P_n(x)$, then we expect the rule to be exact for any degree $\leq n$.

For the open/closed rules, we looked for $\int_a^b f(x) dx \approx \sum_{j=1}^n c_j f(x_j)$, which gives n free parameters $\{c_1, c_2, \dots, c_n\}$.

For Gaussian Quadrature, we will learn both $\{c_j\}_{j=1}^n$ and $\{x_j\}_{j=1}^n$, so we can expect the rule to be exact up to $(2n - 1)$

Example: find the Gaussian Quadrature with $n = 2$:

$$\int_{-1}^1 f(x) dx \approx c_1 f(x_1) + c_2 f(x_2)$$

Testing $f(x)$:

- for $f(x) = 1$:

$$\rightarrow \text{LHS} = \int_{-1}^1 dx = 2$$

$$\rightarrow \text{RHS} = c_1 + c_2$$

Assuming this rule to be exact for degree-0 gives:

$$c_1 + c_2 = 2$$

- for $f(x) = x$:

$$\rightarrow \text{LHS} = \int_{-1}^1 x dx = 0$$

$$\rightarrow \text{RHS} = c_1 x_1 + c_2 x_2$$

Assuming this rule to be exact for degree-0 gives:

$$c_1 x_1 + c_2 x_2 = 0$$

- for $f(x) = x^2$:

$$\rightarrow \text{LHS} = \int_{-1}^1 x^2 dx = \frac{2}{3}$$

$$\rightarrow \text{RHS} = c_1 x_1^2 + c_2 x_2^2$$

Assuming this rule to be exact for degree-0 gives:

$$c_1 x_1^2 + c_2 x_2^2 = \frac{2}{3}$$

- for $f(x) = x^3$:

$$\rightarrow \text{LHS} = \int_{-1}^1 x^3 dx = 0$$

$$\rightarrow \text{RHS} = c_1 x_1^3 + c_2 x_2^3$$

Assuming this rule to be exact for degree-0 gives:

$$c_1 x_1^3 + c_2 x_2^3 = 0$$

This gives the system:

$$\begin{cases} c_1 + c_2 = 2 \\ c_1 x_1 + c_2 x_2 = 0 \\ c_1 x_1^2 + c_2 x_2^2 = \frac{2}{3} \\ c_1 x_1^3 + c_2 x_2^3 = 0 \end{cases}$$

The solution to this system is:

$$\begin{aligned} x_1 &= \frac{1}{\sqrt{3}} \\ x_2 &= -\frac{1}{\sqrt{3}} \\ c_1 &= 1 \\ c_2 &= 1 \end{aligned}$$

$$\therefore \int_{-1}^1 f(x) dx \approx f\left(\frac{1}{\sqrt{3}}\right) + f\left(-\frac{1}{\sqrt{3}}\right)$$

Note: the Legendre polynomials are the set of orthogonal polynomials for the interval $[-1, 1]$

Theorem. Suppose $x_1, \dots, x_n$ are the roots of the n^{th} degree Legendre Polynomials and for each $j = 1, \dots, n$:

$$c_j = \int_{-1}^1 \prod_{k=1, k \neq j}^n \frac{x - x_k}{x_j - x_k} dx$$

Then, for any polynomial $p(x)$ of degree $p \leq 2n - 1$, we have:

$$\int_{-1}^1 p(x) dx = \sum_{j=1}^n c_j p(x_j)$$

ie. the Gaussian Quadrature is exact for degree $\leq 2n - 1$.

Remark: the Legendre Polynomials have the properties:

- all roots are real and distinct
- all roots are within $(-1, 1)$
- if $\tilde{x}$ is a root $\implies -\tilde{x}$ is a root

To apply GQ to $[a, b]$: map $\int_a^b f(x) dx$ to $[-1, 1]$.

$$t = \frac{2}{b-a}x - \frac{b+a}{b-a}$$

Hence, when $x = a \implies t = -1$ and $x = b \implies t = 1$.

$$\therefore dx = \frac{b-a}{2} dt$$

So:

$$\int_{-1}^1 f(x) dx = \left(\frac{b-a}{2}\right) \int_{-1}^1 f\left(\frac{b-a}{2}t + \frac{b+a}{2}\right) dt$$

For example, to calculate $\int_0^1 e^x dx (= e^2 - 1)$ using a 2-part GQ:

$$\begin{aligned} \int_0^2 e^x dx &= \frac{2-0}{2} \int_{-1}^1 \exp\left(\frac{2-0}{2}t + \frac{2+0}{2}\right) dt \\ &= \int_{-1}^1 \exp(t+1) dt \\ &\approx \exp\left(1 - \frac{1}{\sqrt{3}}\right) + \exp\left(1 + \frac{1}{\sqrt{3}}\right) \end{aligned}$$

The theorem above says that GQ is exact up to degree $2n - 1$. To show it has $2n - 1$ precision, we can show it fails for some degree $2n$. Take the counterexample:

$$\tilde{P}(x) = \prod_{j=1}^n (x - x_j)^2$$

7 Linear Algebra

7.1 Norms

Let $\mathbb{R}^n$ be the set of all n -dimensional column vectors. To define a sense of distance in $\mathbb{R}^n$, we use the *norm*.

Definition. A **vector norm** on $\mathbb{R}^n$ is a function $\|\cdot\| : \mathbb{R}^n \mapsto \mathbb{R}$ with the following properties:

- (i) $\|\mathbf{x}\| > 0$ for all $\mathbf{x} \in \mathbb{R}^n$
- (ii) $\|\mathbf{x}\| = 0$ if and only if $\mathbf{x} = \mathbf{0}$
- (iii) $\|\alpha\mathbf{x}\| = |\alpha| \|\mathbf{x}\|$ for all $\alpha \in \mathbb{R}$ and $\mathbf{x} \in \mathbb{R}^n$
- (iv) $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

Definition. The l_2 and l_∞ norms for the vector $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ are defined by:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

$$\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$$

Theorem. (Cauchy-Schwarz Inequality) For each $\mathbf{x} = (x_1, x_2, \dots, x_n)^T$ and $\mathbf{y} = (y_1, y_2, \dots, y_n)^T$ in $\mathbb{R}^n$:

$$\mathbf{x}^T \mathbf{y} = \sum_{i=1}^n x_i y_i \leq \sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2} = \|\mathbf{x}\|_2 \cdot \|\mathbf{y}\|_2$$

7.2 Eigenvalues and Eigenvectors

Definition. If A is a square matrix, the **characteristic polynomial** of A is defined by:

$$p(\lambda) = \det(A - \lambda I)$$

Definition. If p is the characteristic polynomial of a matrix A , then the zeroes of p are known as **eigenvalues**. If λ is an eigenvalue of A , and $\mathbf{x} \neq \mathbf{0}$ satisfies $(A - \lambda I)\mathbf{x} = \mathbf{0}$ then $\mathbf{x}$ is an **eigenvector** of A , corresponding to the eigenvalue λ .

If $\mathbf{x}$ is the e-vector associated with the real e-value λ , then $A\mathbf{x} = \lambda\mathbf{x}$

7.3 Power Method

Assume:

- $A = A^T \in \mathbb{R}^{m \times m}$ (symmetric real matrix)
- let λ_j for $j \in \{1, 2, \dots, m\}$ be the eigenvalues
- let q_j for $j \in \{1, 2, \dots, m\}$ be the orthonormal eigenvectors
- the “Rayleigh Quotient” is $r(x) := \frac{x^T A x}{\|x\|^2}$. It is scalar and has the property that $r(q_j) = \lambda_j$

7.3.1 Rayleigh Quotient

Given some non-zero vector in $\mathbb{R}^m$, $r(x)$ gives a scalar value that is ‘eigenvalue-like’ in the l^2 least squares sense. For an eigenvalue λ , we know by definition $Ax - \lambda x = 0$, so we want to try and perform $\min_{\alpha} \|Ax - \alpha x\|$ for some scalar α . The normal equation is $x^T(Ax - \alpha x) = 0$ Rearranging this equation gives:

$$\alpha := \frac{x^T A x}{\|x\|^2}$$

Thus:

$$r(x) = \min_{\alpha} \|Ax - \alpha x\|$$

So, if we obtain an approximation to an eigenvector, the Rayleigh quotient should give us a ‘stable’ or ‘robust’ estimate of the eigenvalue.

Some properties of the Rayleigh Quotient:

→ The Rayleigh Quotient is continuous and differentiable:

$$\nabla_x r(x) = \frac{2}{\|x\|^2} (Ax - r(x)x)$$

→ eigenvectors $x = q_j$ are stationary points of the Rayleigh Quotient.

→ if we restrict to the unit sphere $\|x\| = 1$, then $r(x) = x^T A x$ is continuous with stationary points at the eigenvectors

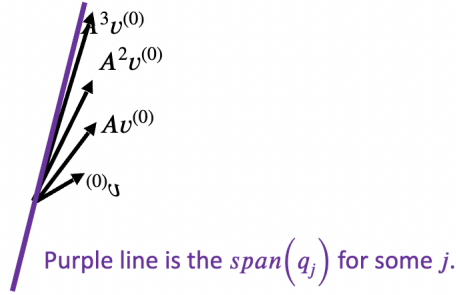
→ by Taylor’s Theorem, $r(x) = r(q_j) + \mathcal{O}(\|x - q_j\|^2)$ for x near q_j , and assuming the eigenvectors are simple (isolated on the sphere)

→ thus, $r(x) = \lambda_j + \mathcal{O}(\|x - q_j\|^2)$ for x near q_j , leading to a quadratically accurate approximation

7.3.2 Power Iteration

If we have an approximation for an eigenvector (for a symmetric matrix), we can use the Rayleigh Quotient to approximate the corresponding eigenvalue.

Let $v^{(0)}$ be a unit vector. If we repeatedly apply A to $v^{(0)}$, we observe that the vectors tend to be aligned with some eigenvector of A .



A may shrink or stretch a generic vector $v^{(0)}$, so we rescale the iterates to produce unit vectors.

The Power Iteration algorithm is:

```

for  $k = 1, 2, \dots$  do
     $w = Av^{(k-1)}$ 
     $v^{(k)} = \frac{w}{\|w\|}$ 
     $\lambda^{(k)} = (v^{(k)})^T Av^{(k)}$ 
end

```

First, we approximate an eigenvector, and second approximate its eigenvalue. In this form, the eigenvalue can be computed at the end.

If this is to be coded, a stopping condition can be introduced:

```

while  $\|\tilde{r}\| < \epsilon$  do
     $w = Av^{(k-1)}$ 
     $v^{(k)} = \frac{w}{\|w\|}$ 
     $\lambda^{(k)} = (v^{(k)})^T Av^{(k)}$ 
     $\tilde{r} = Av^{(k)} - \lambda^{(k)}v^{(k)}$ 
end

```

The stopping is there to check that the residual is small ie. how close the pair is to being an actual eigenvalue / eigenvector. We could also use:

$$\frac{\|\tilde{r}\|}{\|\lambda^{(k)}v^{(k)}\|} < \epsilon$$

Theorem. If $|\lambda_1| \geq |\lambda_2| \geq \dots \geq |\lambda_m| \geq 0$ and $q_1^T v^{(0)} \neq 0$, then:

$$\|v^{(k)} - (\pm q_1)\|_2 \leq C_1 \left| \frac{\lambda_2}{\lambda_1} \right|^k$$

$$|\lambda^{(k)} - \lambda_1| \leq C_2 \left| \frac{\lambda_2}{\lambda_1} \right|^{2k}$$

for some positive C_1, C_2 .

The Rayleigh Quotient converges quadratically to the eigenvalue while the eigenvector converges linearly - this gives one eigenpair.

To get the full spectrum (all eigenpairs), assume $|\lambda_1| > |\lambda_2| > \dots > |\lambda_m| > 0$ and use **deflation**.

7.3.3 Deflation

Once we have an eigenvector approximation, say p_1 , we can ‘deflate’ the matrix A by removing the eigenvector.

One way is to define A' as:

$$A' = A - \lambda_1 p_1 p_1^T$$

Note: $A'p_1 = 0$ and for any other eigenvector ($i \neq 1$), $A'p_i = \lambda_i p_i$. Thus, A' has the same eigenpairs as A except (λ_1, p_1) . We can then repeat the power method for A' and find more eigenpairs.

7.4 Linear Models

The main problem in Linear Algebra is of the sort: find $\vec{x} \in \mathbb{R}^n$ such that that $A\vec{x} = \vec{b}$, where $A \in \mathbb{R}^{n \times n}$ and $\vec{b} \in \mathbb{R}^n$.

We can write $A = [\vec{a}_1 \ \vec{a}_2 \ \dots \ \vec{a}_n]$ ($\vec{a}_i$ is a column vector in $\mathbb{R}^n$). So the correct way to think about the problem is to decompose $\vec{b} \in \mathbb{R}^n$ with respect to the column space of A :

$$\vec{b} = x_1 \vec{a}_1 + x_2 \vec{a}_2 + \dots + x_n \vec{a}_n$$

where x_1 through x_n are the weights.

If we think in terms of applications:

$$\begin{aligned} \vec{b} &= A\vec{x} \\ &= [\vec{a}_1 \ \vec{a}_2 \ \dots \ \vec{a}_n] \vec{x} \end{aligned}$$

Here, each row is a data sample, each column is a feature, and $\vec{x}$ is the weight vector (the importance for each feature).

Some hands-on examples:

- Parkinson's telemonitoring: $\vec{b}$ is the total score $\in [0, 266]$, a measure of the progression of the disability. Columns of A are different features, like age, sex, audio/visual measures etc., and each row represents the data for a different patient.
- Skillcraft: $\vec{b}$ is the rank of a player. The columns (features) are age, hours per week, total hours, actions per minute, etc.

Let $A \in \mathbb{R}^{m \times n}$ (m samples of data, n features). If we want $A\vec{x} = \vec{b}$ ($\vec{x} \in \mathbb{R}^n$, $\vec{b} \in \mathbb{R}^m$)

- Case 1: if $m > n$, more samples/data than features, we end up with a system that is 'underparametrized' or 'overdetermined' (more equations than unknowns). It is possible then that no solution exists.
- Case 2: if $m < n$, more features than samples/data, we have a system that is 'overparametrized' or 'underdetermined' (more unknowns than equations). For this, no unique solution exists.

Instead of solving the system $A\vec{x} = \vec{b}$, we define the **residual** $\vec{r} = \vec{b} - A\vec{x}$, $\vec{r} \in \mathbb{R}^m$, $\vec{x} \in \mathbb{R}^n$ so that:

$$\min_{\vec{x} \in \mathbb{R}^n} \|\vec{r}\|_2^2 \iff \min_{\vec{x} \in \mathbb{R}^n} \|\vec{b} - A\vec{x}\|_2^2 \quad (\text{Case 1})$$

$$\min_{A\vec{x}=\vec{b}} \|\vec{x}\|_2^2 \quad (\text{Case 2})$$

This is known as the 'least squares problem' (LS)

To minimize the LS problem, set $f(\vec{x}) = \|\vec{b} - A\vec{x}\|_2^2$, $f: \mathbb{R}^n \mapsto \mathbb{R}^+$. The critical point is given by $\nabla_{\vec{x}} f(\vec{x}) = \vec{0}_n$:

$$(A^T A)\vec{x} = A^T \vec{b}$$

This is known as the 'Normal Equation' (NE), which is an $(n \times n)$ linear system.

Theorem. The least squares problem has at least one solution $\vec{x}^*$. If $\vec{x}$ is another solution to the problem, then $A\vec{x} = A\vec{x}^*$ (same output), and the residual vector $\vec{r}^* = \vec{b} - A\vec{x}^*$ is unique.

The LS problem is after rescaling by $\frac{1}{2}$:

$$\begin{aligned} & \min_{\vec{x} \in \mathbb{R}^n} \frac{1}{2} \|\vec{b} - A\vec{x}\|_2^2 \\ \implies & \min_{\vec{x} \in \mathbb{R}^n} \frac{1}{2} \sum_i (b_i - A_i \vec{x})^2 \end{aligned}$$

Let $f(\vec{x}) = \frac{1}{2} \sum_i (b_i - A_i \vec{x})^2$. We get:

$$\nabla_{\vec{x}} f(\vec{x}) = A^T (A\vec{x} - \vec{b}) \in \mathbb{R}^n$$

and the minimum is the critical point when:

$$\nabla_{\vec{x}} f(\vec{x}^*) = \vec{0}_n$$

To solve the LS problem, we will apply an FPM to the root finding problem.

Let $\vec{x}^{(k)}$ be our iterator.

$$\begin{aligned} \vec{x}^{(k+1)} &= \vec{x}^{(k)} - \tau \nabla_{\vec{x}} f(\vec{x}^{(k)}) \\ &= \vec{x}^{(k)} - \tau \left(A^T A \vec{x}^{(k)} - A^T \vec{b} \right) \\ &= (I_{n \times n} - \tau A^T A) \vec{x}^{(k)} + \tau A^T \vec{b} \end{aligned}$$

where $\tau > 0$ is a parameter. We need $0 < \tau < \frac{2}{\lambda_{\max}(A^T A)}$

This is known as the **GRADIENT DESCENT PROBLEM**.

7.5 Non-linear Models

Som issues arising from the linear model:

- we are fixing A ie. the features
- linear functions are not expressive - sometimes we need something better than linear to classify

In order to get more expressiveness (ie. be able to approximate non-linear, higher-dimensional systems), we will look at deep networks.

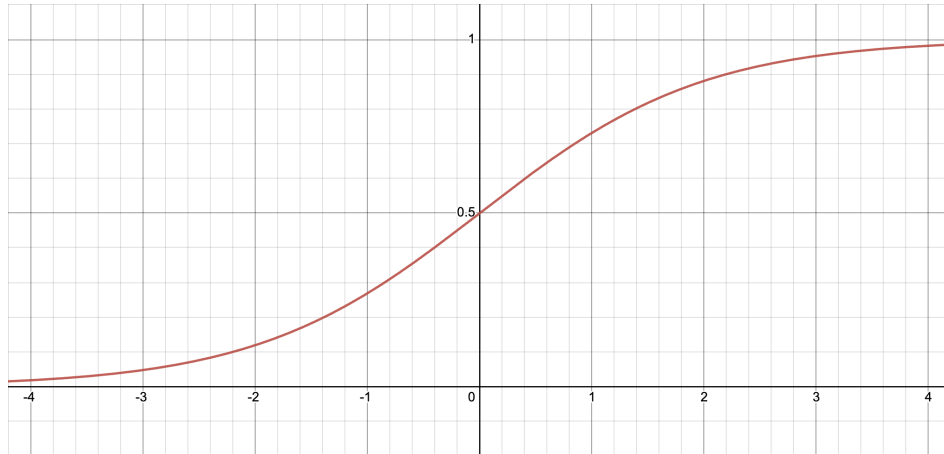
We classify non-linearity in deep networks using:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

for $z \in \mathbb{R}$. It has the derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

We plot $\sigma(z)$:



It can be considered to be a sort of ‘on-off switch’ for classification.

We can build function approximations using combinations of σ functions with different linear inputs: $f(z) \approx \sum$ step functions.

So mathematically, we construct linear layers by:

$$W\vec{x} + \vec{b}$$

W is the weight vector, $\vec{x}$ is the input vector to a layer, and $\vec{b}$ is the bias vector.

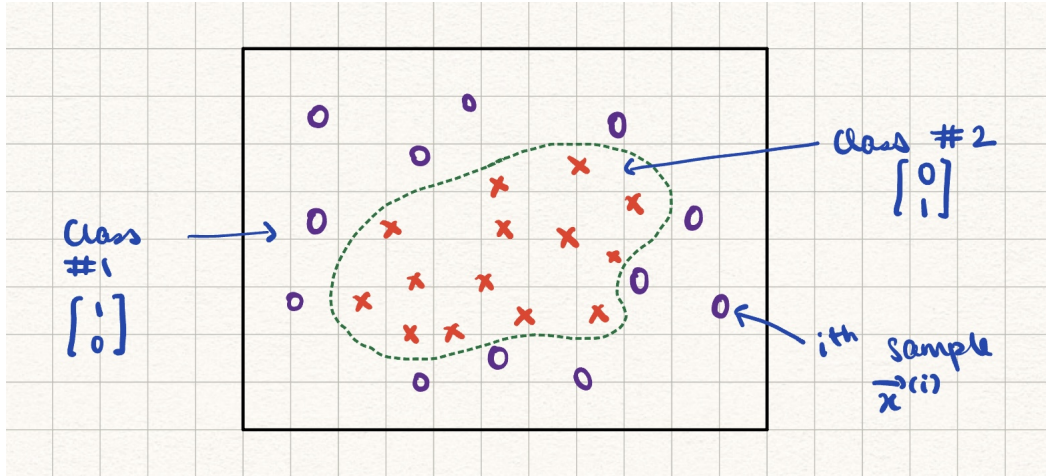
We “activate” the linear layer by applying the function $\sigma(z) = \frac{1}{1+e^{-z}}$ element-wise.

$$\vec{z} \in \mathbb{R}^n \implies \sigma(\vec{z}) = \begin{bmatrix} \sigma(z_1) \\ \vdots \\ \sigma(z_n) \end{bmatrix} \in \mathbb{R}^n$$

That is:

$$\sigma(W\vec{x} + \vec{b}) \in \mathbb{R}^n$$

Example: 4-layer, 2-classifier.



From the image above, we can infer that an input (i^{th} sample) to the classifier system would be of the form:

$$\vec{x}^{(i)} = \begin{bmatrix} x_1^{(i)} \\ x_2^{(i)} \end{bmatrix}$$

similarly, an output from the classifier would be:

$$y^{(i)} = \begin{cases} \begin{pmatrix} 1 \\ 0 \end{pmatrix} & \text{if } \vec{x}^{(i)} \in \text{Class 1} \\ \begin{pmatrix} 0 \\ 1 \end{pmatrix} & \text{if } \vec{x}^{(i)} \in \text{Class 2} \end{cases}$$

The inner layers ('hidden' layers) of this network are characterized by $\vec{h}^{[l]}$, where l is the layer it is a part of. $\vec{h}^{[1]}$ will simply be the input into the network, ie. $\vec{x}$, while $\vec{h}^{[4]}$ will be the final output from the network, ie. $F(\vec{x}) \approx \vec{y}$.

The general workflow of this network is:

$$\underbrace{\mathbb{R}^2}_{\text{input}} \longrightarrow \underbrace{\mathbb{R}^2}_{2 \text{ hidden neurons}} \longrightarrow \underbrace{\mathbb{R}^3}_{3 \text{ hidden neurons}} \longrightarrow \underbrace{\mathbb{R}^2}_{\text{output}}$$

This gives:

$$\rightarrow \vec{h}^{[1]} = \vec{x}$$

$$\rightarrow \vec{h}^{[2]} = \sigma \left(W^{[2]} \vec{h}^{[1]} + \vec{b}^{[2]} \right)$$

$$\rightarrow \vec{h}^{[3]} = \sigma \left(W^{[3]} \vec{h}^{[2]} + \vec{b}^{[3]} \right)$$

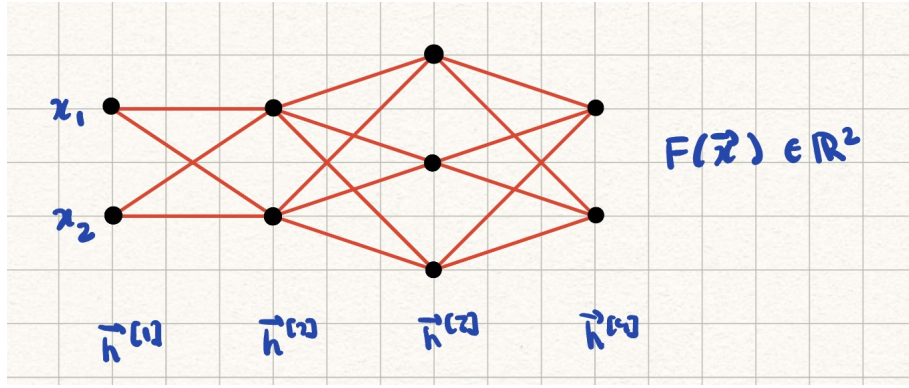
The last (output) layer is then:

$$F(\vec{x}) = \vec{h}^{[4]} = \sigma \left(W^{[4]} \vec{h}^{[3]} + \vec{b}^{[4]} \right)$$

This can be expanded to:

$$F(\vec{x}) = \sigma \left(W^{[4]} \sigma \left(W^{[3]} \sigma \left(W^{[2]} \vec{x} + \vec{b}^{[2]} \right) + \vec{b}^{[3]} \right) + \vec{b}^{[4]} \right)$$

As a graph/network, this looks like:



Each edge is of the form $W_{a,b}^{[l]}$ (l is the layer it ends in, a is the node # in the ending layer, b is the node # in the starting layer).

The number of edges in the graph is the number of trainable weights (weight matrices), and the number of vertices after the first layer is the number of biases. This gives a total of $16 + 7 = 23$ trainable parameters (compared to a single fixed A in a linear model).

We define a cost function:

$$\begin{aligned} \text{Cost} &= \text{Cost} \left(W^{[2]}, W^{[3]}, W^{[4]}, \vec{b}^{[2]}, \vec{b}^{[3]}, \vec{b}^{[4]} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \frac{1}{2} ||\vec{y}^{(i)} - F(\vec{x}^{(i)})|| \\ &= \frac{1}{n} \sum_{i=1}^n \tilde{C}_{\vec{x}^{(i)}} \end{aligned}$$

Let $\vec{p}$ be the vector of all trainable parameters:

$$\vec{p} = \begin{bmatrix} W_{1,1}^{[2]} & W_{1,2}^{[2]} & \dots & \vec{b}_2^{[4]} \end{bmatrix}$$

Using the FPM defined by Gradient Descent, we update $\vec{p}$ by:

$$\vec{p}^{(k+1)} = \vec{p}^{(k)} - \tau \nabla_{\vec{p}} \text{Cost}(\vec{p}^{(k)})$$

Instead of a full FPM (gradient descent), we use Stochastic GD. We randomly pick an $i \in \{1, 2, \dots, n\}$:

$$\vec{p}^{(k+1)} = \vec{p}^{(k)} - \tau \nabla_{\vec{p}} \tilde{C}_{\vec{x}^{(i)}}(\vec{p}^{(k)})$$